

Bachelorarbeit

Entwicklung eines Virtio-Grafikkartentreibers für hhuOS.

vorgelegt von

Safi Al Saloum

aus Syrien, Idlib

Abteilung Betriebssysteme
Prof. Dr. Michael Schöttner
Heinrich-Heine-Universität Düsseldorf

13. April 2026

Erstgutachter:	Prof. Dr. Fabian Ruhland
Zweitgutachter:	Prof. Dr. Martin Mauve
Betreuer:	Prof. Dr. Fabian Ruhland

Inhaltsverzeichnis

1 Einleitung **1**
- 1.1 Allgemeine Einleitung . . . 1
- 1.2 Hintergrund zu Betriebssystemen und Virtualisierung . . . 2
- 1.3 Virtio-Standard und seine Bedeutung . . . 2
- 1.4 Problemstellung . . . 3
- 1.5 Forschungsfragen . . . 3
- 1.6 Forschungsziele . . . 4
- 1.7 Bedeutung der Forschung . . . 4

2 Theoretischer Hintergrund und verwandte Arbeiten **5**
- 2.1 Virtualisierungskonzepte und die x86-Architektur . . . 5
  - 2.1.1 Architektonische Privilegien: VMX-Root und Non-Root . . . 5
  - 2.1.2 Speicheradressübersetzung: Der verschachtelte Durchlauf (Nested Walk) . . . 6
- 2.2 VirtIO-Architektur und Standardisierung . . . 6
  - 2.2.1 Die Initialisierungs-Zustandsmaschine . . . 6
- 2.3 Der Virtqueue-Mechanismus: Tiefergehende Betrachtung . . . 8
  - 2.3.1 Deskriptorverkettung und Gather-Write . . . 8
  - 2.3.2 Speicherreihenfolge und der Fence-Mechanismus . . . 8
- 2.4 VirtIO-GPU-Protokoll und 2D-Rendering-Pipeline . . . 8
  - 2.4.1 Der Lebenszyklus eines 2D-Befehls . . . 9
- 2.5 Vergleich virtualisierter Grafiklösungen . . . 9

3 Systemanalyse und Treiberentwurf **12**
- 3.1 Analyse der Systemanforderungen . . . 12
  - 3.1.1 Funktionale Anforderungen: Die Befehlspipeline . . . 12
  - 3.1.2 Nicht-funktionale Anforderungen: Sicherheit und Leistung . . . 12
- 3.2 Gesamtarchitektur des Treibers . . . 13
- 3.3 Zentrale Datenstrukturen . . . 14
  - 3.3.1 Der Gerätezustand (VirtioGpuDevice) . . . 14
  - 3.3.2 Die Split-Virtqueue-Strukturen . . . 15
  - 3.3.3 GPU-Steuerpakete . . . 15
- 3.4 Detaillierter Initialisierungsablauf . . . 15
  - 3.4.1 Phase 1: Erkennung und Capability-Mapping . . . 16
  - 3.4.2 Phase 2: Der Handshake . . . 16
- 3.5 Entwurf der Hardwareabstraktionsschicht (HAL) . . . 16
- 3.6 Speicherverwaltung: Die semantische Lücke . . . 16
  - 3.6.1 Direct Memory Access (DMA) und Pinnen . . . 16

3.7 Virtqueue-Entwurf und Synchronisation . . . 17
3.7.1 Deskriptorverkettung und Anordnung . . . 17
3.7.2 Der Synchronisations-Handshake . . . 17
3.7.3 Der Benachrichtigungszyklus im Überblick . . . 18
3.7.4 Synchrone Wait-Used-Logik . . . 19
3.8 Fazit des Systementwurfs . . . 19

4 Implementierung des VirtIO-GPU-Treibers 20
4.1 Geräteerkennung und Probe-Mechanismus . . . 20
4.1.1 PCI-Enumeration und Identifikation . . . 20
4.1.2 Capability-Durchlauf und Abbildung . . . 20
4.2 Der VirtIO-Handshake und die Initialisierung . . . 21
4.2.1 Feature-Verhandlung . . . 21
4.2.2 Statusübergänge . . . 22
4.3 Virtqueue-Einrichtung und DMA-Verwaltung . . . 22
4.3.1 Speicherausrichtung und Allokation . . . 22
4.4 Das Control-Command-Protokoll . . . 23
4.4.1 Vorab allozierte Befehlspuffer . . . 23
4.4.2 Signalisierung und Polling . . . 23
4.5 Framebuffer und Ressourcenlebenszyklus . . . 24
4.5.1 Die 2D-Ressourcen-Pipeline . . . 24
4.6 Dynamisches Rendering, Animationsthread und Laufzeit-Umschalter . . . 24
4.6.1 Per-Frame-Renderer und Dirty-Rect-Präsentation . . . 25
4.6.2 Dedizierter Animationsthread . . . 25
4.6.3 Laufzeit-Moduswechsel per Tastatur . . . 26
4.6.4 Geänderte Quelldateien im Überblick . . . 27
4.7 Implementierung der HAL-Schnittstellenschicht . . . 27
4.8 Fazit der Implementierung . . . 27

5 Testen und Validierung 28
5.1 Methodischer Rahmen . . . 28
5.1.1 Instrumentierung und Messung . . . 28
5.2 Testfall 1: Speicherkonsistenz (statischer Modus – Schachbrettmuster) . . . 29
5.3 Testfall 2: Dynamisches Per-Frame-Rendering (animierter Farbverlauf mit HUD) . . . 29
5.4 Testfall 3: Laufzeit-Moduswechsel (Taste C) . . . 30
5.5 Spezifikation der validierten Schnittstelle . . . 32
5.5.1 Wichtige Strukturen und Typen . . . 32
5.5.2 VirtIO-GPU Befehle und Antworten . . . 32
5.5.3 Queue-Konfiguration . . . 33
5.5.4 Schlüsselfunktionen . . . 33
5.6 Quantitative Leistungsanalyse . . . 33
5.6.1 Der „Trap and Emulate“-Engpass . . . 33
5.7 Qualitative Stabilität und visuelle Artefakte . . . 35
5.8 Diskussion der aktuellen Einschränkungen . . . 35
5.9 Zukunftsausblick . . . 36

6 Zusammenfassung und zukünftige Arbeiten 37

6.1 Zusammenfassung der Forschung und Beiträge 37
6.2 Technische Reflexion: Paravirtualisierung vs. Emulation 37
6.3 Architektonische Auswirkungen auf hhuOS 38
6.4 Einschränkungen und offene Herausforderungen 38
6.5 Zukünftige Arbeiten: In Richtung 3D und darüber hinaus 38
6.6 Abschließende Bemerkungen 39

Literatur **40**

Abbildungsverzeichnis **41**

Tabellenverzeichnis **42**

Kapitel 1

Einleitung

1.1 Allgemeine Einleitung

Die Entwicklung des modernen Rechnens ist im Wesentlichen eine Geschichte des Übergangs von physikalischen Zwängen zu logischen Abstraktionen. In der Pionierzeit der Informatik war Software eine maßgeschneiderte Kreation, die speziell für die einzigartigen elektrischen und mechanischen Eigenschaften einer einzelnen Maschine entwickelt wurde. Die Aufgabe des Programmierers bestand darin, jedes Register, jede Speicheradresse und jede Spannungsänderung manuell zu verwalten. Mit zunehmender Komplexität der Rechenaufgaben wuchs jedoch auch die Notwendigkeit einer vermittelnden Schicht – des Betriebssystems (Operating System, OS). Wie von Silberschatz et al. [1] beschrieben, entwickelte sich das Betriebssystem zu einem Ressourcenverwalter, der die rohe, flüchtige Natur der Hardware in stabile, wiederverwendbare Dienste abstrahiert.

In der heutigen Landschaft hat diese Abstraktion durch Virtualisierung ihren Höhepunkt erreicht. Hardware wird nicht länger als statische physische Einheit betrachtet, sondern als dynamische, softwaredefinierte Ressource. In diesem Paradigma hat sich die grafische Benutzeroberfläche (Graphical User Interface, GUI) zum primären Medium für die Mensch-Computer-Interaktion (Human-Computer Interaction, HCI) entwickelt. Während frühe Systeme auf einfache textbasierte Fernschreiber angewiesen waren, erwarten moderne Benutzer und Entwickler gleichermaßen hochauflösende, reaktionsschnelle grafische Ausgaben. Infolgedessen hat sich die Grafikprozessoreinheit (Graphics Processing Unit, GPU) von einem peripheren Luxus zu einem erstklassigen Bürger der Computerarchitektur entwickelt. Dieser Übergang zeigt sich besonders deutlich in der Verschiebung hin zu hardwarebeschleunigten Desktop-Compositoren, bei denen selbst die grundlegende Fensterverwaltung eine direkte Interaktion mit der Grafikhardware erfordert – ein Prozess, der in virtualisierten Umgebungen einen erheblichen Mehraufwand verursacht [2].

Diese Arbeit konzentriert sich auf die entscheidende Schnittstelle dieser beiden Trends: die Entwicklung eines Grafiktreibers in einer virtualisierten Umgebung für einen experimentellen Betriebssystemkern, insbesondere hhuOS/hhuOS. Wir untersuchen den „VirtIO-GPU“-Standard – eine paravirtualisierte Schnittstelle, die entwickelt wurde, um Gastsystemen hochleistungsfähige Grafik bereitzustellen. Durch die Implementierung dieses Treibers für einen benutzerdefinierten Kernel überbrücken wir die Kluft zwischen abstrakter Kernellogik und sichtbarer grafischer Ausgabe. Diese Arbeit, die auf den etablierten Forschungszielen der HHU-Betriebssystemgruppe aufbaut [3], ist eine Untersuchung darüber, wie moderne Paravirtualisierungsstandards genutzt werden können, um die Hardwarein-

teraktion zu vereinfachen und gleichzeitig eine Leistung zu erhalten, die sich nativen Geschwindigkeiten annähert.

1.2 Hintergrund zu Betriebssystemen und Virtualisierung

Um die Rolle eines VirtIO-GPU-Treibers zu verstehen, muss man die technischen Hürden kennen, die die Virtualisierung zu überwinden versucht. Das Konzept der Virtualisierung geht auf die 1960er Jahre mit IBM-Großrechnern zurück, aber seine Anwendung auf die x86-Architektur war über Jahrzehnte hinweg bekanntermaßen schwierig. Der x86-Befehlssatz war ursprünglich nicht gemäß den klassischen Popek- und Goldberg-Anforderungen virtualisierbar konzipiert. Diese Anforderungen besagen, dass alle „sensiblen“ Befehle einen Trap zum Hypervisor auslösen müssen, wenn sie von einem Gast ausgeführt werden.

Auf Standard-x86-Hardware läuft das Betriebssystem typischerweise im „Ring 0“, während Anwendungen im „Ring 3“ ausgeführt werden. In einer virtualisierten Umgebung „glaubt“ das Gast-Betriebssystem, es sei in Ring 0, wird aber tatsächlich von einem Hypervisor verwaltet. Frühe Lösungen nutzten „Binary Translation“, um Gastcode spontan umzuschreiben. Darauf folgte die Einführung der hardwaregestützten Virtualisierung (Intel VT-x und AMD-V), die die notwendigen Hardware-Primitiven bereitstellte, um Gast- und Host-Modi effektiv zu unterstützen.

Trotz der Hardwareunterstützung bleibt die „Vollständige Emulation“ langsam. In diesem Modell simuliert eine VMM wie QEMU eine gesamte physische Maschine in Software. Für ein Gast-Betriebssystem erscheint die emulierte Hardware als ein Standard-Legacy-Gerät, wie z.B. eine Cirrus Logic VGA-Karte. Wie Belkhiri und Dagenais in ihrer Leistungsstudie [2] anmerken, muss die CPU jedes Mal, wenn das Gast-Betriebssystem ein Pixel ändert, einen „VM-Exit“ auslösen und den Kontext zum Host wechseln. Dieser „Trap-and-Emulate“-Zyklus erzeugt einen massiven Engpass für Grafiken. Die „Semantische Lücke“ – das mangelnde Verständnis des Hosts darüber, was der Gast zu erreichen versucht – verschärft dies zusätzlich, da der Host nur rohe Speicherschreibvorgänge sieht und keine hochwertigen Zeichenbefehle.

1.3 Virtio-Standard und seine Bedeutung

Der VirtIO-Standard, der 2022 auf Version 1.2 aktualisiert wurde [4], stellt den Versuch der Industrie dar, paravirtualisierte Schnittstellen zu vereinheitlichen. VirtIO bietet eine standardisierte Transportschicht über den PCI-Bus und ermöglicht es einem einzigen Treiber, mit jedem Hypervisor zu arbeiten, der die Spezifikation unterstützt. Diese Standardisierung ist entscheidend für das „Cloud-Zeitalter“, in dem virtuelle Maschinen zwischen verschiedenen Host-Architekturen migrierbar sein müssen, ohne die Gerätefunktionalität zu verlieren.

Das Herzstück der VirtIO-Architektur ist der „Virtqueue“. Ein Virtqueue ermöglicht Zero-Copy- oder Datentransfer mit geringem Overhead unter Verwendung von drei Teilen:

1. **Deskriptortabelle:** Eine Liste von Speicherpuffern, die vom Gast-Betriebssystem bereitgestellt werden.

2. **Available Ring:** Hier platziert der Gasttreiber Indizes von Deskriptoren, die vom Host verarbeitet werden sollen.

3. **Used Ring:** Hier benachrichtigt der Host (Hypervisor) den Gast, dass die Verarbeitung abgeschlossen ist.

Die Bedeutung der VirtIO-GPU-Spezifikation liegt insbesondere in ihrer Abkehr vom synchronen „Register-Memory"-Modell. Herkömmliche VGA-Geräte verlassen sich darauf, dass die CPU manuell Pixeldaten in MMIO-Bereiche schreibt, was die CPU anhält, bis die Bus-Transaktion abgeschlossen ist. VirtIO-GPU verwendet ein befehlsbasiertes Protokoll, das hochrangige Befehle in einen Virtqueue sendet. Dies ermöglicht einen asynchronen Betrieb, der für die Systemreaktionsfähigkeit und eine vorhersagbare Kommunikation zwischen VMs entscheidend ist [5]. Moderne Versionen der Spezifikation führen auch „Packed Virtqueues" ein, die die Effizienz von hochauflösenden Frame-Übertragungen verbessern.

1.4 Problemstellung

Trotz der Eleganz des VirtIO-Standards bringt seine Implementierung in einem experimentellen Kernel wie hhuOS/hhuOS mehrere erhebliche Herausforderungen mit sich:

1. Die Speicherabbildungslücke: Es gibt drei Ebenen von Speicheradressen: Gast-Virtual (GVA), Gast-Physisch (GPA) und Host-Physisch (HPA). Das VirtIO-GPU-Gerät benötigt GPA für seine Virtqueues. Die Koordinierung der internen Seitentabellen des Kernels mit den Anforderungen des PCI-Busses ist eine ständige Quelle von „Seitenfehlern" (Page Faults).

2. Debugging-Probleme in der frühren Initialisierungsphase: Die Entwicklung eines Grafiktreibers ist besonders in den frühen Phasen der Initialisierung anspruchsvoll, da Fehler häufig bereits vor der ersten erfolgreichen Bildausgabe auftreten. Wenn der Treiber beispielsweise während der Feature-Verhandlung oder beim Aufbau der Virtqueues scheitert, bleibt die grafische Ausgabe aus. Die Analyse solcher Fehler erfolgt daher typischerweise über serielle Protokollierung und weitere nicht-grafische Debugging-Verfahren.

3. Asynchrone Synchronisation: Wenn der Gast einen Puffer ändert, während der Host ihn für eine Bildschirmaktualisierung liest, kommt es zu visuellen Störungen (Corruption). Die Implementierung eines präzisen Benachrichtigungssystems innerhalb einer Mikrokernel-Architektur erfordert ein tiefes Verständnis der Scheduler-Latenz, ein Thema von zunehmender Bedeutung in modernen sicheren eingebetteten Systemen [6].

4. Protokollstrenge: Die VirtIO 1.1/1.2-Spezifikationen sind sehr spezifisch in Bezug auf die Statusbits (ACKNOWLEDGE, DRIVER, FEATURES_OK). Jede Abweichung führt dazu, dass der Hypervisor das Gerät einfriert.

1.5 Forschungsfragen

- **RQ1:** Wie kann ein Treiber eine strikte Trennung zwischen generischer VirtIO-Logik und der OS-spezifischen Hardwareabstraktionsschicht (Hardware Abstraction Layer, HAL) aufrechterhalten?

- **RQ2:** Welche Leistungsauswirkungen haben paravirtualisierte Befehlswarteschlangen im Vergleich zu Legacy-VGA in einem Forschungskernel?

- **RQ3:** Wie kann der Kernel DMA-gestützte Ressourcen für die GPU unter strikter Speicherisolierung sicher verwalten?
- **RQ4:** Was ist die minimal realisierbare Implementierung von VirtIO-GPU, die für eine treibergesteuerte Ausgabe erforderlich ist?

1.6 Forschungsziele

Das primäre Ziel ist der Entwurf und die Implementierung eines funktionalen 2D-VirtIO-GPU-Treibers.

1. **PCI-Infrastruktur:** Aufbau eines Mechanismus zum Durchsuchen des PCI-Busses und zur Abbildung der fünf spezifischen VirtIO-PCI-Fähigkeiten (Capabilities).
2. **Warteschlangenverwaltung:** Aufbau eines robusten Virtqueue-Systems, das die Verkettung von Deskriptoren (Descriptor Chaining) handhabt.
3. **Grafikprotokoll:** Implementierung kritischer 2D-Befehle: `RESOURCE_CREATE_2D`, `RESOURCE_ATTACH_BACKING`, `SET_SCANOUT` und `RESOURCE_FLUSH`.
4. **Verifikation:** Erstellung eines Testmustergenerators innerhalb des Kernels zur visuellen Überprüfung.

1.7 Bedeutung der Forschung

Akademisch gesehen liefert diese Arbeit eine dokumentierte Implementierung eines komplexen paravirtualisierten Geräts. Technisch gesehen verwandelt dieser Treiber das Ziel-Betriebssystem von einem rein textbasierten Projekt in eine visuell leistungsfähige Plattform. Durch die Verwendung von VirtIO stellen wir die Kompatibilität mit modernen Cloud-Infrastrukturen und Hochdurchsatz-Kommunikationsframeworks sicher [5] und bieten eine Vorlage für zukünftige VirtIO-basierte Treiber in sicheren Umgebungen [6].

Kapitel 2

Theoretischer Hintergrund und verwandte Arbeiten

2.1 Virtualisierungskonzepte und die x86-Architektur

Virtualisierung ist der architektonische Prozess der Entkopplung physischer Hardware von den darauf ausgeführten Softwareebenen. Im Kontext des hhuOS-Forschungskernels ist Virtualisierung nicht nur eine Kompatibilitätsschicht, sondern ein Sicherheitsprimitiv, das eine starke Isolierung zwischen Gerätetreibern und der Kernlogik des Kernels ermöglicht.

2.1.1 Architektonische Privilegien: VMX-Root und Non-Root

Die x86-Architektur erfüllte ursprünglich die Virtualisierungsanforderungen von Popek und Goldberg nicht, da bestimmte Befehle sich je nach Privilegienstufe unterschiedlich verhielten, ohne Traps auszulösen. Dies wurde mit Intel VT-x durch die Einführung von VMX (Virtual Machine Extensions) behoben.

Der Prozessor arbeitet in zwei sich gegenseitig ausschließenden Modi:

1. **VMX-Root-Betrieb:** Im Allgemeinen für den Hypervisor reserviert. Die CPU hat die volle Kontrolle über den Systemzustand und kann physische Register manipulieren.

2. **VMX-Non-Root-Betrieb:** Die Umgebung für das Gast-Betriebssystem (hhuOS). Selbst wenn der Gastkernel in Ring 0 läuft, lösen sensible Befehle (z. B. `HLT`, `CPUID` oder MMIO-Zugriffe) einen **VM-Exit** aus.

Die Leistung eines VirtIO-GPU-Treibers ist untrennbar mit der Häufigkeit dieser Übergänge verbunden. Jeder VM-Exit erfordert, dass der Prozessor den Gastzustand in der VMCS (Virtual Machine Control Structure) speichert, was über 1.500–3.000 Taktzyklen verbraucht. Ein wichtiges Entwurfsziel von VirtIO ist es, Befehle zu bündeln, um diese Übergänge zu minimieren.

2.1.2 Speicheradressübersetzung: Der verschachtelte Durchlauf (Nested Walk)

Die Virtualisierung führt eine zusätzliche Ebene der Speicheradressierung ein. In hhuOS verwaltet der Treiber Gast-Virtual-Adressen (GVA), die über die zweistufigen Seitentabellen des Gasts in Gast-Physische-Adressen (GPA) übersetzt werden. Für den tatsächlichen Zugriff auf den Arbeitsspeicher wird die GPA anschließend über die Extended Page Tables (EPT) des Hosts in eine Host-Physische-Adresse (HPA) überführt.

Wie in Abbildung 2.1 gezeigt, ergibt sich damit ein verschachtelter Übersetzungsvorgang:

$$(GVA \rightarrow GPA)_{\text{Gast}} \times (GPA \rightarrow HPA)_{\text{Host}}$$

Dieser zusätzliche Übersetzungsschritt erhöht den Aufwand der Speicheradressierung. Für GPU-Treiber ist daher die DMA-Verwaltung besonders wichtig, da alle an das VirtIO-GPU-Gerät übergebenen Puffer stabil im Gast-Physikraum referenzierbar sein müssen.

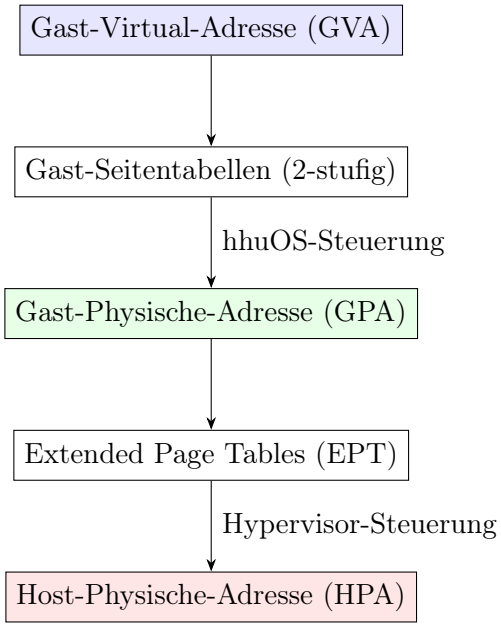


Abbildung 2.1: Verschachtelte Speicherübersetzung in einer virtualisierten 32-Bit-Umgebung

2.2 VirtIO-Architektur und Standardisierung

VirtIO wurde entwickelt, um paravirtualisierte E/A zu standardisieren. Durch die Verwendung eines standardisierten Busses (PCI) und einer gemeinsamen Ringpufferstruktur (Virtqueues) wird die Notwendigkeit für die VMM beseitigt, Hunderte von Legacy-Gerätefehlern zu emulieren.

2.2.1 Die Initialisierungs-Zustandsmaschine

Die VirtIO 1.2-Spezifikation schreibt ein streng sequenzielles Initialisierungsprotokoll vor. Dies stellt sicher, dass Gast und Host eine synchronisierte Sicht auf die Gerätefähigkeiten haben, bevor irgendwelche MMIO- oder DMA-Operationen stattfinden. Der

Fortschritt wird durch das **Device Status Register** gesteuert, ein 8-Bit-Feld in der gemeinsamen Konfigurationsstruktur [4].

Schritt-für-Schritt-Initialisierungsprotokoll

Der Treiber muss die folgenden Zustände durchlaufen. Das Überspringen eines Schritts oder das Schreiben einer ungültigen Bitsequenz löst die Bedingung `DEVICE_NEEDS_RESET` aus.

1. **Reset (0x00):** Nach der Entdeckung schreibt der Treiber 0 in das Statusregister. Dies löscht alle alten Zustände im Backend und setzt die Virtqueues zurück.

2. **ACKNOWLEDGE (0x01):** Der Gast benachrichtigt den Host, dass er das Gerät bemerkt hat. Dies ist das erste „Ich bin da"-Signal.

3. **DRIVER (0x02):** Der Gast zeigt an, dass er einen passenden Treiber (den hhuOS-VirtIO-GPU-Treiber) besitzt und in der Lage ist, das Gerät zu verwalten.

4. **FEATURES_OK (0x08):** Dies ist die kritischste Phase. Der Treiber liest die `device_features` (eine 32-Bit- oder 64-Bit-Bitmaske, die vom Host bereitgestellt wird) und schreibt die `driver_features` zurück (die Teilmenge, die er tatsächlich unterstützt). Nach dem Schreiben liest der Treiber das Statusregister erneut, um sicherzustellen, dass der Host die Teilmenge akzeptiert hat.

5. **Virtqueue-Einrichtung:** Während des Zustands `FEATURES_OK`, aber vor dem Setzen von `DRIVER_OK`, muss der Treiber physischen Speicher für die Virtqueues allozieren und ihre physischen Adressen in das Konfigurations-BAR schreiben.

6. **DRIVER_OK (0x04):** Das endgültige „Los"-Signal. Erst nachdem dieses Bit gesetzt ist, kann das Gerät mit der Verarbeitung von Deskriptoren aus dem Available Ring beginnen.

Feature-Verhandlung und der Version-1.1+-Handshake

Für einen modernen Treiber in hhuOS ist die Verhandlung von `VIRTIO_F_VERSION_1` zwingend erforderlich. Dieses Bit signalisiert, dass das Gerät der modernen Spezifikation (Little-Endian-Zugriff, speicherabgebildete Fähigkeiten) folgt und nicht dem Legacy-Standard 0.9.5.

Der Ausfallsicherungsmechanismus: DEVICE_NEEDS_RESET

Die Zustandsmaschine enthält eine Sicherung gegen asynchrone Fehler. Wenn der Hypervisor auf einen schwerwiegenden Backend-Fehler stößt (z.B. eine ungültige DMA-Adresse oder einen Protokollverstoß), setzt er das **DEVICE_NEEDS_RESET** (0x40)-Bit. Der hhuOS-Treiber muss dieses Bit regelmäßig abfragen oder während der Interruptbehandlung überprüfen. Sobald es gesetzt ist, wird das Gerät zu einem „Zombie" – es ignoriert alle weiteren Benachrichtigungen, bis die Initialisierungssequenz bei 0x00 neu gestartet wird [4], [7].

Auswirkungen auf die hhuOS-Integration

In der hhuOS-Implementierung wird diese Zustandsmaschine innerhalb der Methode `VirtioGpuDevice::init()` implementiert. Eine häufige Fehlerquelle, die in dieser Arbeit behandelt wird, ist der Zeitpunkt der Virtqueue-Allokation. Gemäß der Spezifikation müssen die Host-Physischen-Adressen (HPA) der Deskriptortabellen in das gemeinsame Konfigurations-BAR geschrieben werden, *nachdem* die Feature-Verhandlung abgeschlossen ist, aber *bevor* `DRIVER_OK` gesetzt wird. Die Nichteinhaltung dieser Reihenfolge führt dazu, dass der Host die „Türklingel"-Benachrichtigungen (Doorbell) ignoriert, da das Backend noch nicht weiß, welche Speicherbereiche es auf Befehle überwachen soll.

2.3 Der Virtqueue-Mechanismus: Tiefergehende Betrachtung

Der Virtqueue ist die Hochleistungs-Transportschicht. In hhuOS implementieren wir den Split Virtqueue, der aus drei verschiedenen Speicherbereichen besteht, die an der Cache-Linie ausgerichtet sein müssen (normalerweise 64 Byte).

2.3.1 Deskriptorverkettung und Gather-Write

VirtIO-GPU-Befehle erstrecken sich oft über mehrere nicht zusammenhängende Speicherseiten. Dies wird durch Deskriptorverkettung (Descriptor Chaining) gehandhabt. Jeder Deskriptor enthält einen `NEXT`-Zeiger und ein `FLAGS`-Feld.

- **VIRTQ_DESC_F_NEXT:** Zeigt an, dass der Befehl im nächsten Deskriptoreintrag fortgesetzt wird.
- **VIRTQ_DESC_F_WRITE:** Zeigt an, dass der Speicher für das Gerät zum Schreiben (Antworten) bestimmt ist.

2.3.2 Speicherreihenfolge und der Fence-Mechanismus

Da der Host (Backend) und der Gast (Frontend) separate Ausführungskontexte sind, ist die Speichersichtbarkeit nicht garantiert. 1. Der Gast schreibt den Befehl in die Deskriptortabelle. 2. Der Gast führt eine Store-Store-Barriere (`sfence` auf x86) aus, um sicherzustellen, dass die Tabelle aktualisiert ist, bevor das Gerät sie sieht. 3. Der Gast aktualisiert den Index des Available Ring. 4. Der Gast „tritt" (kick) das Gerät, indem er in das Notify-BAR schreibt.

2.4 VirtIO-GPU-Protokoll und 2D-Rendering-Pipeline

Das VirtIO-GPU-Gerät ist im Wesentlichen ein befehlsbasierter Stream-Prozessor. Im Gegensatz zu einer VGA-Karte, bei der man in einen Framebuffer schreibt, sendet man bei VirtIO-GPU Anweisungen zur Manipulation von Ressourcen (Resources).

2.4.1 Der Lebenszyklus eines 2D-Befehls

Der Lebenszyklus eines Frames in hhuOS wird durch fünf primäre Befehle gesteuert:

Dieses „Transfer-then-Flush"-Modell ist der Kern der Effizienz von VirtIO-GPU. Es erlaubt dem Gast, Tausende von lokalen Speicherschreibvorgängen ohne VM-Exits durchzuführen und nur dann zum Hypervisor zu wechseln, wenn ein vollständiger Frame zur Anzeige bereit ist [2], [4].

2.5 Vergleich virtualisierter Grafiklösungen

Die folgende Tabelle fasst die während der architektonischen Phase der hhuOS-Entwicklung untersuchten Kompromisse zusammen.

Tabelle 2.1: Taxonomie der Grafikvirtualisierungstechniken

Merkmal	**Emulierte VGA**	**SR-IOV / Passthrough**	**VirtIO-GPU**
Mechanismus	Trap-and-Emulate	Hardware-Partitionierung	Paravirtualisierung
VM-Exits	Einer pro Pixel-Schreibzugriff	Minimal	Einer pro Flush
Treiberanforderung	Generisch (VGA)	Nativer Hersteller-treiber	VirtIO-Standard
Portabilität	Universell	Hardwaregebunden	Spezifikationsbasiert

Wie von Silberschatz et al. [1] angemerkt, bietet Passthrough zwar die höchste Leistung, verletzt jedoch das grundlegende Mikrokernel-Prinzip der Hardwareabstraktion. VirtIO-GPU bietet eine „semantische Brücke" – der Host versteht, dass der Gast eine „Flush"-Operation durchführt, was Optimierungen wie hostseitige Hardwareskalierung oder Komprimierung ermöglicht, die mit reiner VGA-Emulation nicht möglich sind.

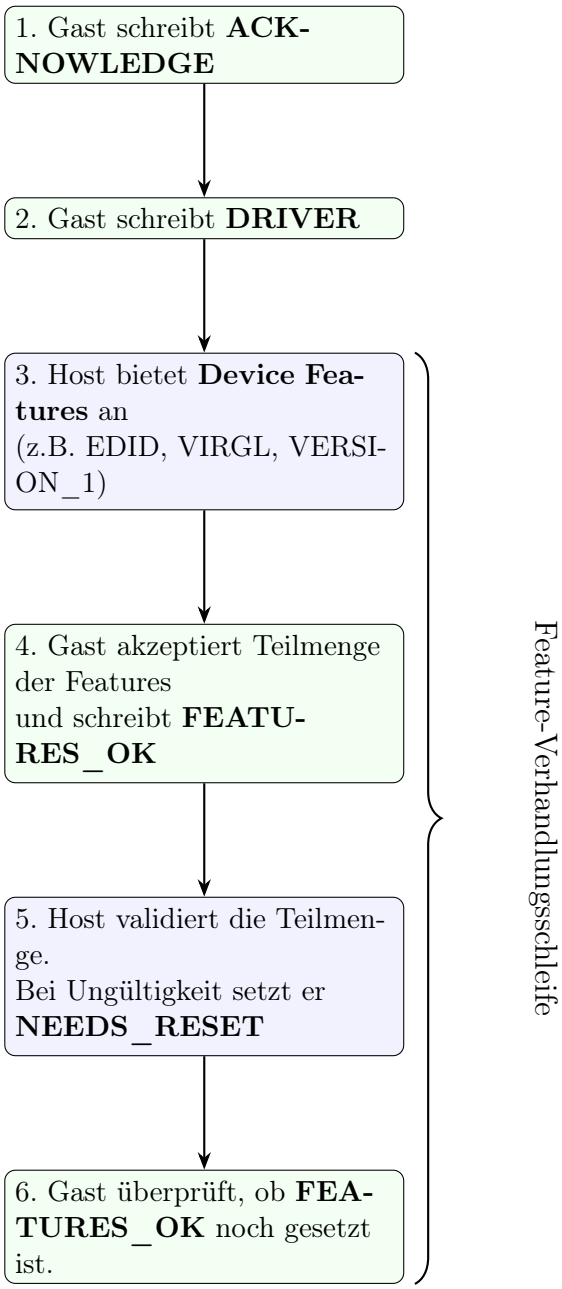


Abbildung 2.2: Detaillierte Logik der Feature-Verhandlung

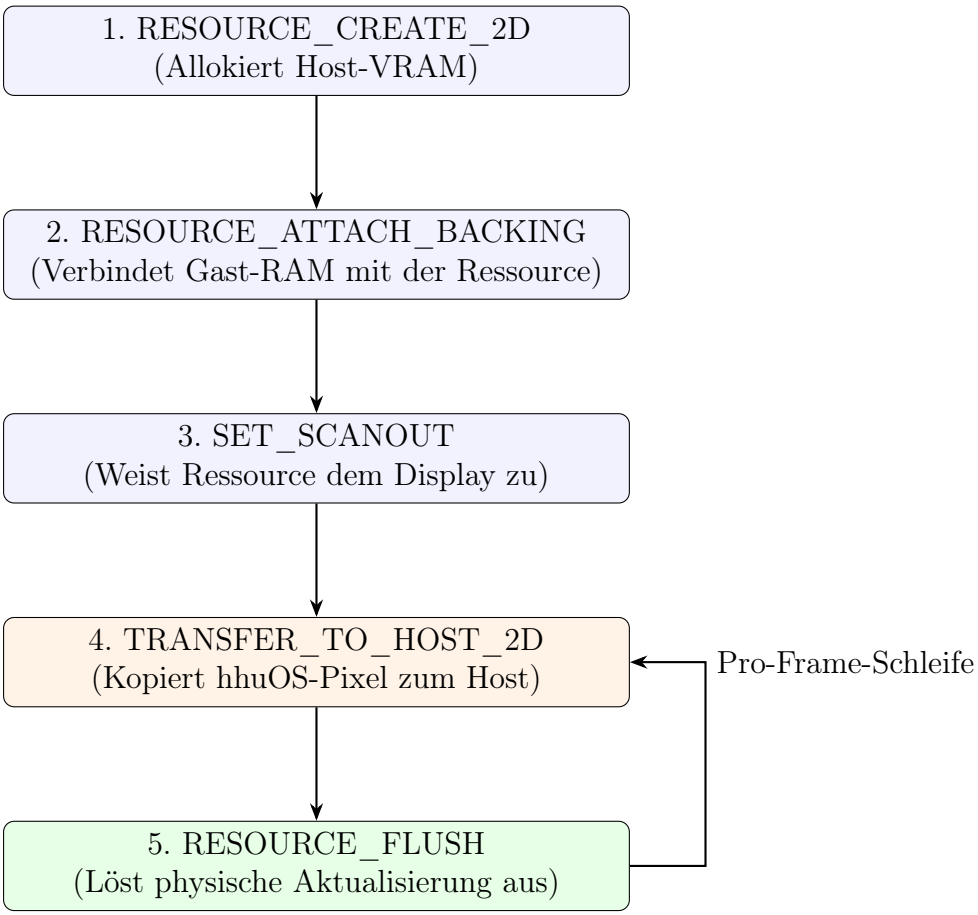


Abbildung 2.3: Flussdiagramm der VirtIO-GPU-2D-Rendering-Pipeline

Kapitel 3

Systemanalyse und Treiberentwurf

3.1 Analyse der Systemanforderungen

Die Integration eines leistungsfähigen Grafiktreibers in den hhuOS-Mikrokernel bringt im Vergleich zu monolithischen Umgebungen wie Linux eine Reihe besonderer Herausforderungen mit sich. In hhuOS besitzt der Treiber keinen inhärenten privilegierten Zugriff auf den Systemspeicher; stattdessen muss er Ressourcen explizit von den Speicher- und PCI-Diensten des Kernels anfordern.

3.1.1 Funktionale Anforderungen: Die Befehlspipeline

Der Treiber muss einen bidirektionalen Kommunikationskanal zwischen dem Gast-Betriebssystem und der Host-VMM bereitstellen.

1. **PCI-Bus-Enumeration:** Der Treiber muss mit dem hhuOS-PCI-Manager interagieren, um das VirtIO-GPU-Gerät (ID `0x1050`) zu lokalisieren und seine herstellerspezifischen Capabilities zu analysieren, damit die MMIO-Offsets für die Konfiguration bestimmt werden können.

2. **2D-Ressourcenverwaltung:** Der Treiber muss die Logik implementieren, um hostseitige Texturen (Resources) zu erstellen, Gast-RAM-Seiten als Backing Stores anzuhängen und den Lebenszyklus dieser Ressourcen zu verwalten, um Speicherlecks im Hypervisor zu verhindern.

3. **Synchrone und asynchrone E/A:** Während Frame-Aktualisierungen (*Flushes*) asynchron ablaufen können, erfordern Steuerbefehle, etwa das Erstellen einer Ressource, einen synchronen Handshake, um sicherzustellen, dass die Ressourcen-ID vor der weiteren Verwendung gültig ist.

3.1.2 Nicht-funktionale Anforderungen: Sicherheit und Leistung

1. **Strikte Isolierung:** Gemäß der Mikrokernel-Philosophie [8] arbeitet der Treiber in seinem eigenen Adressraum. Fehler in der Behandlung des VirtIO-GPU-Protokolls müssen auf den Treiberprozess beschränkt bleiben, um eine systemweite Instabilität zu verhindern.

2. **Interrupt-Effizienz:** In einer virtualisierten Umgebung sind Interrupts (VM-Exits) teuer. Der Entwurf muss, wo immer möglich, MSI-X nutzen und für Hochdurchsatz-Szenarien den Polling-Modus unterstützen, um den Overhead von Kontextwechseln zu minimieren.

3.2 Gesamtarchitektur des Treibers

Der hhuOS-VirtIO-GPU-Treiber ist als modulares System konzipiert, das hochrangige Grafikaufrufe von den Interaktionen mit der zugrunde liegenden Hardware entkoppelt. Diese Modularität ist in Mikrokernel-Umgebungen, in denen Dienstgrenzen strikt eingehalten werden, von entscheidender Bedeutung.

Die Architektur gliedert sich in drei primäre Funktionsbereiche:

1. **Frontend-API-Schicht:** Diese Schicht stellt die Schnittstelle zum restlichen hhuOS-Ökosystem dar. Sie übersetzt generische Zeichen- oder Aktualisierungsanfragen in das logische VirtIO-GPU-Protokoll. Darüber hinaus verwaltet sie den virtuellen Framebuffer und stellt sicher, dass Pixeldaten auf Anwendungsebene korrekt für die GPU formatiert sind.

2. **Zentrale VirtIO-Engine:** Dies ist das Herzstück des Treibers. Sie enthält die Protokollhandler, die `VIRTIO_GPU_CMD`-Pakete erzeugen. Außerdem verwaltet sie die beiden obligatorischen Virtqueues: die *Control Queue* für Ressourcenverwaltung und Initialisierung sowie die *Cursor Queue* für hardwarebeschleunigte Mauszeiger.

3. **OS-spezifische HAL- und Schnittstellenschicht:** Diese Schicht bildet die Schnittstelle zu den hhuOS-Kerndiensten. Sie übernimmt das Einblenden von PCI-BARs in den Adressraum des Treibers, die Allokation physisch zusammenhängender DMA-Puffer sowie die Benachrichtigung des Hypervisors über die Notify-Register.

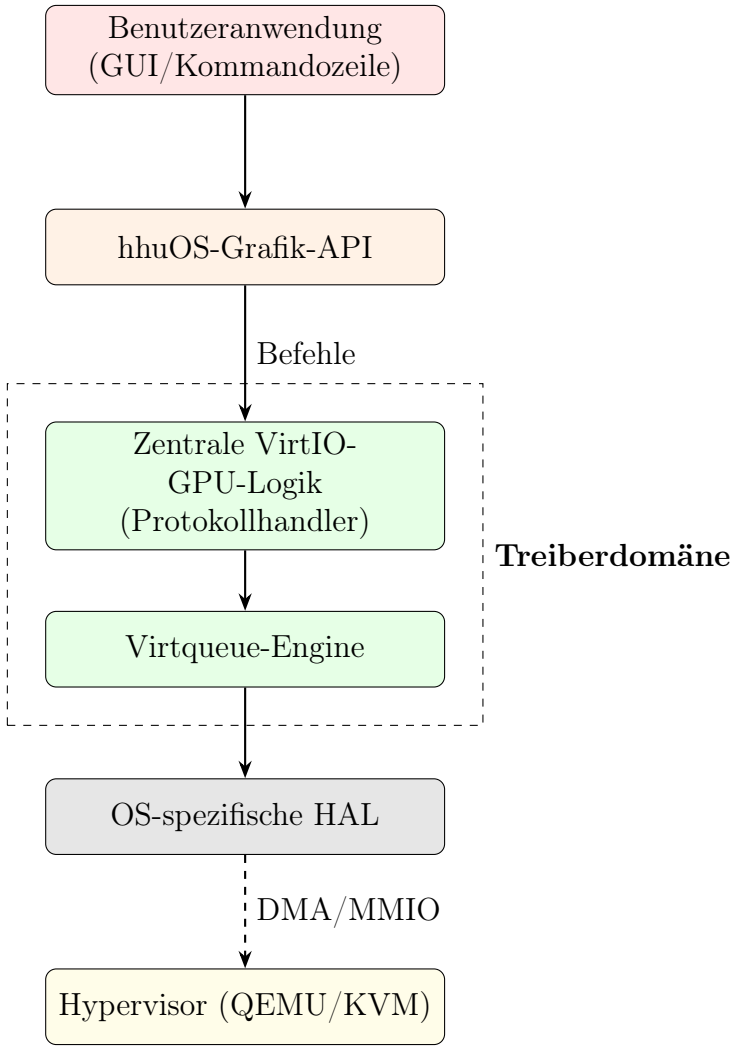


Abbildung 3.1: Mehrschichtige Treiberarchitektur in hhuOS

3.3 Zentrale Datenstrukturen

Die Effizienz des Treibers hängt von Datenstrukturen ab, die präzise mit der VirtIO-1.2-Spezifikation übereinstimmen. Da das Gerät, also der Hypervisor, diese Strukturen direkt über DMA ausliest, sind Speicherausrichtung und Padding von zentraler Bedeutung.

3.3.1 Der Gerätezustand (VirtioGpuDevice)

Die zentrale Klasse `VirtioGpuDevice` verwaltet den Laufzeitzustand der Hardwareverbindung. Sie speichert die MMIO-Zeiger auf die PCI-BARs sowie die Instanzen der Virtqueues.

Listing 3.1: Zentrale Gerätestruktur

```
class VirtioGpuDevice {
    // MMIO-Register
    volatile CommonCfg* common_cfg;
    volatile uint32_t* notify_reg;
```

```
    volatile GpuConfig* gpu_cfg;

    // Virtqueues
    Virtqueue control_vq;
    Virtqueue cursor_vq;

    // Ressourcenverwaltung
    uint32_t current_resource_id;
    DmaBuffer framebuffer;
};
```

3.3.2 Die Split-Virtqueue-Strukturen

Die **Split Virtqueue** besteht aus drei Speicherbereichen, die zwischen Gast und Host gemeinsam genutzt werden. In hhuOS werden diese als ein einziger zusammenhängender DMA-Block alloziert, um die Adressübersetzung zu vereinfachen.

1. **Deskriptortabelle:** Ein Array von 16-Byte-Einträgen, das Pufferadresse, Länge und Flags (z. B. `NEXT`, `WRITE`) definiert.

2. **Available Ring:** Ein vom Gast beschriebener Ringpuffer, der Indizes von Deskriptoren enthält, die zur Verarbeitung bereitstehen.

3. **Used Ring:** Ein vom Host beschriebener Ringpuffer, in dem das Gerät abgeschlossene Aufgaben meldet.

3.3.3 GPU-Steuerpakete

Die gesamte Kommunikation über die Control Queue verwendet einen Standard-Header (`virtio_gpu_ctrl_hdr`). Diese Struktur stellt sicher, dass sich Gast und Host über den Befehlstyp und eventuelle Fehlercodes einig sind.

```
struct virtio_gpu_ctrl_hdr {
    uint32_t type;      // VIRTIO_GPU_CMD_*
    uint32_t flags;
    uint64_t fence_id;
    uint32_t ctx_id;
    uint32_t padding;
};
```

3.4 Detaillierter Initialisierungsablauf

Die Initialisierung eines VirtIO-1.2-Geräts ist eine empfindliche Zustandsmaschinen-Transition. Der hhuOS-Treiber stellt sicher, dass keine E/A versucht wird, bevor der Host das `FEATURES_OK`-Bit bestätigt hat.

3.4.1 Phase 1: Erkennung und Capability-Mapping

Im Gegensatz zu Legacy-PCI-Geräten, die feste BAR-Offsets verwenden, nutzt VirtIO-GPU eine dynamische Liste von Capabilities. Der Treiber durchläuft die verkettete Liste der PCI-Capabilities, um `VIRTIO_PCI_CAP_COMMON_CFG` und `VIRTIO_PCI_CAP_NOTIFY_CFG` zu finden. Dadurch kann der Treiber auch moderne Hypervisoren unterstützen, bei denen die Benachrichtigungsregister in einem anderen BAR liegen als die Konfigurationsregister.

3.4.2 Phase 2: Der Handshake

Der Treiber folgt der in [4] definierten Sequenz:

1. **Reset:** Das Statusregister wird gelöscht, um einen sauberen Ausgangszustand zu gewährleisten.

2. **Feature-Verhandlung:** Der Treiber liest die 64-Bit-Feature-Bitmaske vom Gerät. Er muss `VIRTIO_F_VERSION_1` aushandeln, um moderne 1.1-Features wie Little-Endian-Datenstrukturen zu aktivieren.

3.5 Entwurf der Hardwareabstraktionsschicht (HAL)

Eine wichtige Entwurfsentscheidung war die Implementierung einer dünnen HAL. Diese Schicht abstrahiert die kernelspezifischen Details von hhuOS, zum Beispiel:

- **Speicherabbildung:** Übersetzung physischer PCI-Adressen in den virtuellen Adressraum des Treibers über die Schnittstelle `os_map_mmio()`.

- **Physische Kontiguität:** Sicherstellung, dass DMA-Puffer physisch zusammenhängend sind. In hhuOS wird dies von einem speziellen DMA-Allokator übernommen, der die standardmäßige Heap-Allokation umgeht, um seitenausgerichtete und zusammenhängende Blöcke bereitzustellen.

3.6 Speicherverwaltung: Die semantische Lücke

Einer der komplexesten Aspekte der GPU-Virtualisierung ist die Überbrückung der „semantischen Lücke“. Das Gast-Betriebssystem weiß, welche Pixel es zeichnen möchte, aber die Host-GPU kann nicht direkt auf den privaten Speicher des Gasts zugreifen.

3.6.1 Direct Memory Access (DMA) und Pinnen

Um dieses Problem zu lösen, implementiert der Treiber einen Backing-Store-Mechanismus. Wenn ein Framebuffer in hhuOS alloziert wird, muss der Treiber die entsprechenden Seiten pinnen. Dadurch wird sichergestellt, dass der hhuOS-Kernel diese Seiten nicht auslagert oder neu zuweist, während die Host-GPU daraus liest.

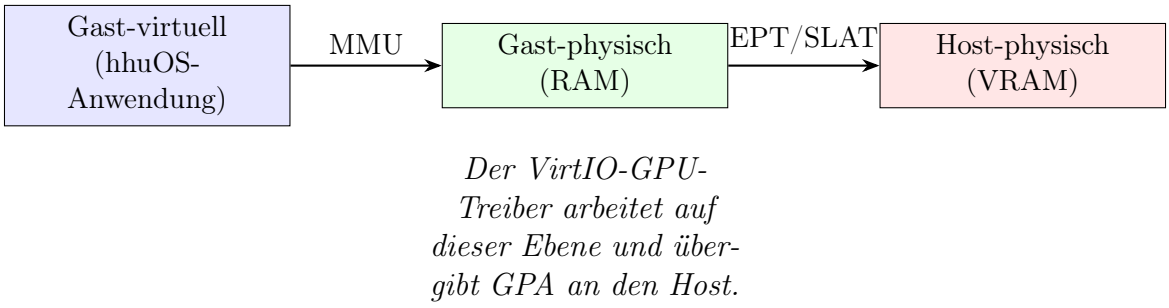


Abbildung 3.2: Dreistufige Adressübersetzung bei VirtIO

3.7 Virtqueue-Entwurf und Synchronisation

Die Virtqueue ist die primäre Abstraktion für asynchrone Datenübertragung in VirtIO. Für den GPU-Treiber wird eine **Split-Virtqueue**-Architektur verwendet. Dieser Entwurf basiert auf einem gemeinsam genutzten Ringpufferprotokoll, das teure VM-Exits durch die Bündelung von Befehlen minimiert.

3.7.1 Deskriptorverkettung und Anordnung

Jeder GPU-Befehl besteht aus einem Anforderungs-Header, der vom Gerät gelesen wird, und einem Antwort-Header, der vom Gerät beschrieben wird. Da sich beide an unterschiedlichen Speicherorten befinden, wird **Deskriptorverkettung** verwendet. Ein einzelner logischer Befehl wird dabei durch zwei oder mehr Deskriptoren dargestellt, die über das `NEXT`-Flag miteinander verknüpft sind.

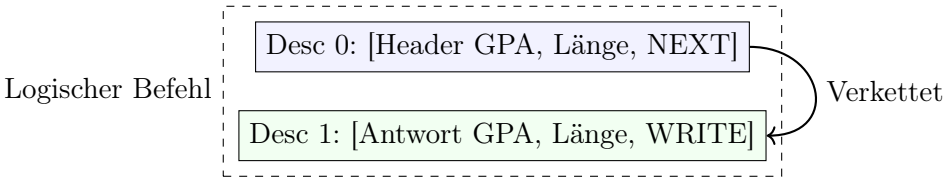


Abbildung 3.3: Virtqueue-Deskriptorverkettung für GPU-Befehle

3.7.2 Der Synchronisations-Handshake

Um die Konsistenz zwischen dem hhuOS-Treiber als Produzent und dem Hypervisor als Konsument zu wahren, wird ein strenges Speicherordnungssprotokoll eingehalten. Dadurch wird verhindert, dass der Host teilweise geschriebene Deskriptoren liest.

1. **Payload schreiben:** Der Treiber schreibt die Befehlsdaten in den DMA-Puffer.
2. **Deskriptor einrichten:** Der Treiber füllt die Deskriptortabelle mit der physischen Adresse des Puffers.
3. **Speicherbarriere (SFENCE):** Ein `SFENCE`-Befehl stellt sicher, dass alle vorherigen Speicheroperationen für andere Prozessoren und die Host-CPU sichtbar sind, bevor der Available-Index aktualisiert wird [4].
4. **Available-Index aktualisieren:** Der Treiber erhöht den Index im Available Ring.

5. **Benachrichtigen (Kick):** Der Treiber schreibt in das PCI-Notify-Offset. Dies löst einen VM-Exit aus und signalisiert dem Host, mit der Verarbeitung zu beginnen.

3.7.3 Der Benachrichtigungszyklus im Überblick

Die Interaktion zwischen der Benachrichtigung durch den Treiber und der späteren Rückmeldung des Geräts wird im Folgenden dargestellt:

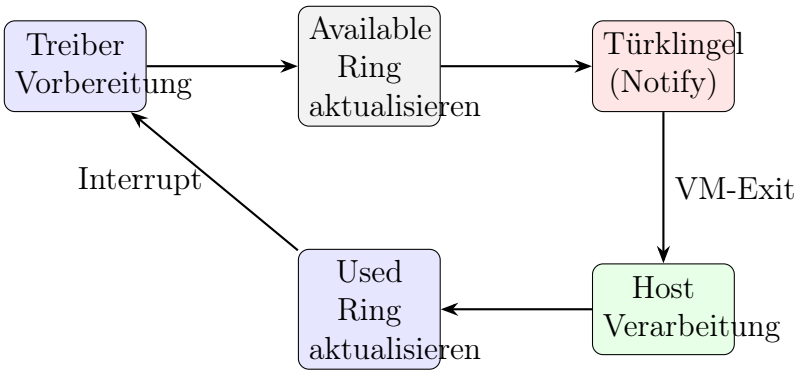
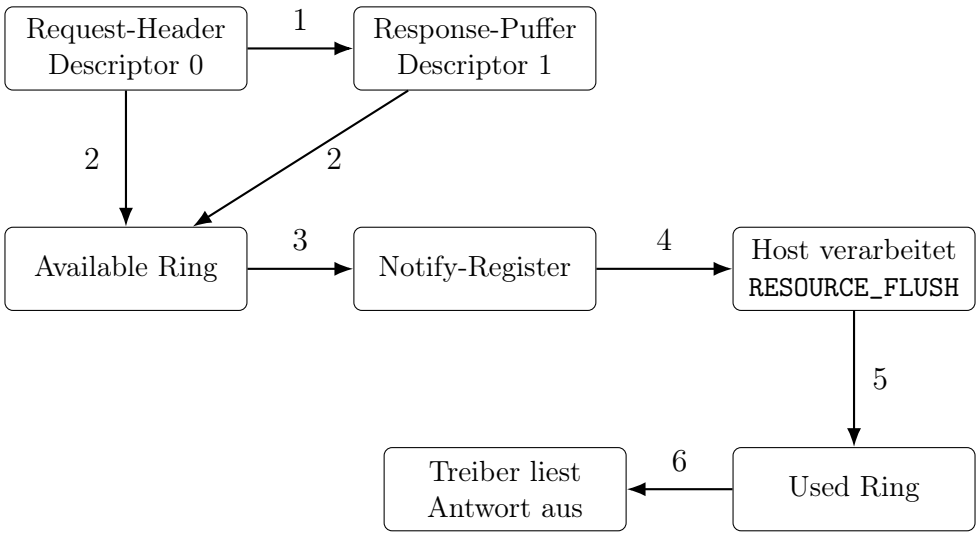


Abbildung 3.4: Asynchroner Befehlslebenszyklus und Signalisierung

Konkretes Beispiel: `RESOURCE_FLUSH` über die Control Queue Abbildung 3.5 zeigt einen beispielhaften Befehlsablauf über die Control Queue am Beispiel von `RESOURCE_FLUSH`. Der Treiber verbindet zunächst Anfrage und Antwort über eine Deskriptorkette, trägt den Kopf der Kette in den Available Ring ein und benachrichtigt den Host über das Notify-Register. Nach der Verarbeitung aktualisiert der Host den Used Ring und stellt die Antwort im vorgesehenen Puffer bereit. Der Treiber kann danach den Abschluss des Befehls auswerten und die weitere Verarbeitung fortsetzen.



Ablauf: (1) Anfrage und Antwort werden über eine Deskriptorkette verbunden. (2) Der Kettenkopf wird in den Available Ring eingetragen. (3) Der Treiber benachrichtigt den Host über das Notify-Register. (4) Der Host verarbeitet den Befehl `RESOURCE_FLUSH`. (5) Der Abschluss wird im Used Ring eingetragen. (6) Der Treiber liest die Antwort aus.

Abbildung 3.5: Beispielhafter Befehlsablauf über die Control Queue am Beispiel von `RESOURCE_FLUSH`

3.7.4 Synchrone Wait-Used-Logik

Obwohl die Virtqueue grundsätzlich asynchron arbeitet, benötigt hhuOS für Einrichtungsbefehle, z. B. `RESOURCE_CREATE_2D`, ein synchrones Verhalten. Der Treiber implementiert daher eine Polling-Schleife, die den `used_idx` des Used Rings überprüft. Um Systemabstürze zu verhindern, ist ein Watchdog-Timer integriert:

$$T_{\text{verstrichen}} > T_{\text{timeout}} \implies \text{Geräte-Reset} \tag{3.1}$$

Wenn der Host den Deskriptor nicht innerhalb von 1000 ms zurückgibt, protokolliert der hhuOS-Kernel einen fatalen Treiberfehler und versucht, die VirtIO-Zustandsmaschine zurückzusetzen.

3.8 Fazit des Systementwurfs

Die in diesem Kapitel durchgeführte Analyse bestätigt, dass das VirtIO-1.2-Protokoll in Kombination mit einer robusten HAL und einer strengen Speicherverwaltung in hhuOS die notwendige Infrastruktur für einen leistungsfähigen Grafikstack bereitstellt. Durch die Betonung der Paravirtualisierung gegenüber der Emulation wird der VM-Exit-Overhead erheblich reduziert und die Grundlage für die in Kapitel 4 beschriebene Implementierung geschaffen.

Kapitel 4

Implementierung des VirtIO-GPU-Treibers

Die Implementierung des VirtIO-GPU-Treibers in hhuOS ist eine mehrschichtige softwaretechnische Aufgabe. Sie überbrückt die Kluft zwischen hochrangigen Grafikaufrufen und der Kommunikation mit der zugrunde liegenden PCI-Hardware. Der Code ist architektonisch in mehrere Submodule unter `src/device/graphic/virtio/` gegliedert. Diese Aufteilung stellt sicher, dass Discovery, Initialisierung, Virtqueue-Transport, Control-Queue-Kommandos, Ressourcenverwaltung und Laufzeitpfade klar voneinander getrennt sind.

4.1 Geräteerkennung und Probe-Mechanismus

Der Lebenszyklus des Treibers beginnt mit der Erkennung der Hardware. In der hhuOS-Umgebung wird dies von der Funktion `virtio_gpu_probe_all_devices()` verwaltet.

4.1.1 PCI-Enumeration und Identifikation

Während des Kernel-Bootvorgangs, insbesondere im Einstiegspunkt `GatesOfHell.cpp` oder über einen Autostart-Hook, wird der PCI-Bus durchsucht. Der Treiber sucht nach der Hersteller-ID `0x1AF4` (Standard für VirtIO) und der Geräte-ID `0x1050` (VirtIO-GPU). Im Gegensatz zu älteren Treibern, die Speicheradressen fest kodieren, liest diese Implementierung dynamisch die Base Address Registers (BARs), um den Konfigurationsraum des Geräts zu lokalisieren.

4.1.2 Capability-Durchlauf und Abbildung

Ein wesentlicher Bestandteil der Implementierung ist die Routine `build_regions_from_caps()`. VirtIO-1.2-Geräte nutzen eine verkettete Liste von PCI-Capabilities, um ihre Registerbereiche zu definieren [4]. Der Treiber durchläuft diese Liste, um die obligatorischen Regionen `COMMON_CFG`, `NOTIFY_CFG` und `ISR_CFG` sowie optional `DEVICE_CFG` zu identifizieren.

Die einzelnen Bereiche erfüllen dabei folgende Aufgaben:

- **Common Configuration:** Wird für die Statusverwaltung, die Feature-Verhandlung und die Auswahl bzw. Konfiguration der Virtqueues verwendet.

- **Notification Region:** Dient als „Türklingel“ (Doorbell), um dem Gerät neue Einträge im Available Ring zu signalisieren.
- **ISR Configuration:** Dient der Rückmeldung von Interrupt- beziehungsweise Completion-relevanten Ereignissen.
- **Device Configuration:** Stellt optionale gerätespezifische Daten wie Display-Parameter bereit.

Für jeden gefundenen Bereich ruft der Treiber die HAL-Funktion `os_map_mmio()` auf, um eine virtuelle Abbildung auf die physischen Hardware-Register zu erzeugen und diese für die weitere Initialisierung im Kernel verfügbar zu machen.

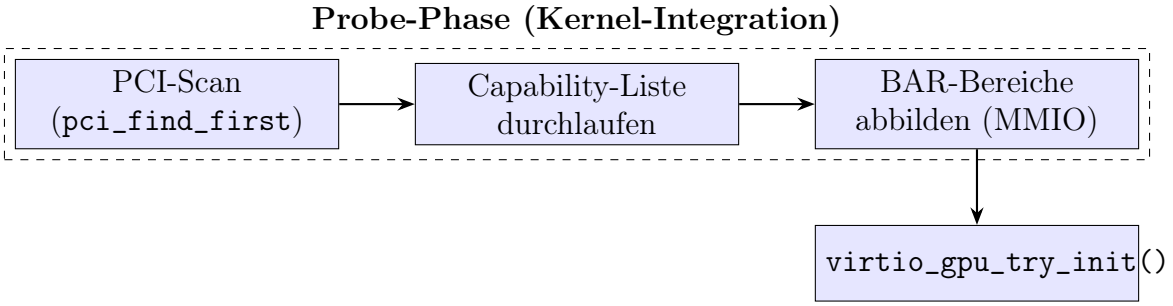


Abbildung 4.1: Detaillierte Treibererkennungs- und Abbildungspipeline

4.2 Der VirtIO-Handshake und die Initialisierung

Nach der Abbildung der MMIO-Bereiche tritt der Treiber in die Funktion `virtio_gpu_try_init()` ein. Diese folgt der strikten Zustandsmaschine der VirtIO-Spezifikation und stellt sicher, dass weder Queue-Zugriffe noch GPU-Kommandos stattfinden, bevor der Host die ausgehandelten Eigenschaften akzeptiert hat.

4.2.1 Feature-Verhandlung

In `negotiate_features()` liest der Treiber die vom Gerät angebotenen Features als 64-Bit-Bitmaske in zwei 32-Bit-Fenstern aus. Anschließend wird konservativ nur die tatsächlich benötigte Teilmenge aktiviert. Zwingend ist dabei `VIRTIO_F_VERSION_1`, da nur damit der moderne VirtIO-Pfad mit Little-Endian-Datenstrukturen und speicherabgebildeten Capabilities genutzt werden kann.

Die Feature-Verhandlung folgt dabei dem üblichen Ablauf:

1. Lesen der vom Gerät angebotenen Feature-Bits,
2. Auswahl der vom Treiber unterstützten Teilmenge,
3. Schreiben der akzeptierten Teilmenge in die Treiberregister,
4. Setzen von `FEATURES_OK`,
5. erneute Validierung, dass das Gerät dieses Bit weiterhin gesetzt lässt.

Zusätzliche Features wie VirGL werden erkannt und intern vermerkt, sind jedoch nicht Bestandteil des aktuellen 2D-Hauptpfads. Die Implementierung verfolgt damit bewusst eine konservative Politik: Stabilität und Kompatibilität haben Vorrang vor einer frühzeitigen Aktivierung optionaler Erweiterungen.

4.2.2 Statusübergänge

Die Implementierung verwendet das Gerätestatusregister, um den Fortschritt der Initialisierung explizit zu signalisieren:

1. **ACKNOWLEDGE:** Der Treiber hat das Gerät erkannt.
2. **DRIVER:** Der Treiber signalisiert, dass er das Gerät grundsätzlich ansteuern kann.
3. **FEATURES_OK:** Gast und Host stimmen dem ausgehandelten Feature-Satz zu.
4. **DRIVER_OK:** Die Initialisierung ist abgeschlossen; das Gerät ist betriebsbereit.

Schlägt ein obligatorischer Schritt fehl, wird der Fehlerstatus gesetzt und die Initialisierung abgebrochen. Dadurch wird verhindert, dass ein teilweise initialisiertes Gerät in einen undefinierten Laufzeitzustand gelangt.

4.3 Virtqueue-Einrichtung und DMA-Verwaltung

Das Kommunikationsrückgrat des Treibers ist die **Split Virtqueue**. Die Funktion `virtqueue_setup()` übernimmt die zentrale Speicherallokation und Konfiguration der Queue-Strukturen.

4.3.1 Speicherausrichtung und Allokation

Da der Hypervisor (QEMU/KVM) direkt auf den RAM des Gasts zugreift, muss der verwendete Speicher physisch zusammenhängend und passend ausgerichtet sein. Der Treiber verwendet `os_alloc_dma_pages()`, um seitenausgerichtete Blöcke zu reservieren und sowohl virtuelle als auch physische Adressen für die weitere Queue-Konfiguration bereitzustellen.

Innerhalb des allokierten DMA-Bereichs werden drei logisch getrennte Strukturen angelegt:

- die **Deskriptortabelle**,
- der **Available Ring**,
- der **Used Ring**.

Die physischen Basisadressen dieser Bereiche werden anschließend in die Common-Configuration-Register des Geräts eingetragen. Erst danach kann die jeweilige Virtqueue aktiviert und für den Befehlsverkehr verwendet werden.

4.4 Das Control-Command-Protokoll

Die GPU wird über Befehle gesteuert, die an die *Control Queue* (Warteschlange 0) gesendet werden. Die Funktion `control_queue_send()` implementiert einen synchronen Wrapper um die grundsätzlich asynchrone Virtqueue-Kommunikation.

Dieser Ansatz erlaubt es der höheren Initialisierungslogik, mit einem einfachen Request-Response-Modell zu arbeiten, während die eigentliche Übertragung weiterhin dem VirtIO-Ringmechanismus folgt.

4.4.1 Vorab allozierte Befehlspuffer

Um Fragmentierung zu vermeiden und die Befehlsübertragung reproduzierbar zu halten, nutzt der Treiber vorab allozierte DMA-Puffer für Requests und Responses. Diese Puffer werden während der Initialisierung der Control Queue reserviert und anschließend für sämtliche Kommandos wiederverwendet.

Die synchrone Übertragung eines GPU-Kommandos erfolgt über genau zwei Deskriptoren:

- einen **Request-Deskriptor** für den Befehl,
- einen **Response-Deskriptor** für die Antwort des Geräts.

Der Request-Puffer wird vom Gerät gelesen, während der Response-Puffer vom Gerät beschrieben wird. Beide Puffer werden über eine Deskriptorkette logisch zu einem gemeinsamen Kommando verbunden.

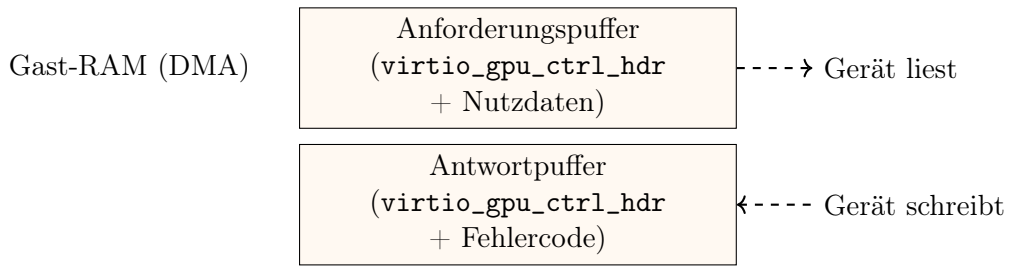


Abbildung 4.2: Speicheraufteilung für Anforderung und Antwort der Control Queue

4.4.2 Signalisierung und Polling

Nachdem Request- und Response-Puffer vorbereitet wurden, veröffentlicht der Treiber den Kopf der Deskriptorkette im Available Ring und benachrichtigt das Gerät über das Notify-Register. Anschließend wartet der Treiber auf den Abschluss im Used Ring.

Der Ablauf ist dabei:

1. Request in den DMA-Puffer kopieren,
2. Response-Puffer vorbereiten beziehungsweise leeren,
3. zwei Deskriptoren aufsetzen und verkettet publizieren,
4. Kettenkopf in den Available Ring eintragen,

5. Gerät per `virtqueue_notify()` benachrichtigen,

6. auf Completion über `virtqueue_wait_used()` warten,

7. Antwort in den Aufruferpuffer übernehmen.

Da die aktuelle Implementierung für Einrichtungs- und Kontrollkommandos ein synchrones Verhalten benötigt, wird eine Polling-Schleife mit Watchdog-Timer verwendet. Antwortet das Gerät nicht innerhalb des vorgegebenen Zeitfensters, wird ein Fehler protokolliert und der betreffende Schritt als fehlgeschlagen behandelt.

4.5 Framebuffer und Ressourcenlebenszyklus

Das eigentliche Ziel des Treibers ist die Bereitstellung eines sichtbaren Framebuffers. Dies wird in `create_framebuffer_resource()` realisiert, wo lokaler DMA-Speicher, VirtIO-GPU-Ressource und Scanout logisch miteinander verbunden werden.

4.5.1 Die 2D-Ressourcen-Pipeline

Die Erstellung und Sichtbarmachung eines Framebuffers folgt einer klaren Befehlskette:

1. **RESOURCE_CREATE_2D:** Allokiert eine hostseitige 2D-Ressource für die angegebene Auflösung.

2. **RESOURCE_ATTACH_BACKING:** Verbindet Gast-RAM-Seiten mit der hostseitigen Ressource.

3. **SET_SCANOUT:** Verbindet die Ressource mit dem virtuellen Monitor.

4. **TRANSFER_TO_HOST_2D:** Überträgt die geänderten Pixelbereiche aus dem Gast-Framebuffer an den Host.

5. **RESOURCE_FLUSH:** Signalisiert dem Host, die aktualisierten Bereiche sichtbar zu machen.

Diese Trennung zwischen lokalem Schreiben in den Gast-Framebuffer und expliziter Präsentation gegenüber dem Host ist zentral für die Effizienz des Treibers. Der Gast kann seine Pixel zunächst vollständig lokal erzeugen, bevor nur die tatsächlich relevanten Bereiche an das Gerät übertragen und sichtbar gemacht werden.

4.6 Dynamisches Rendering, Animationsthread und Laufzeit-Umschalter

Nachdem die grundlegende statische Ausgabe (Schachbrettmuster) erfolgreich validiert wurde, wurde der Treiber um eine vollständige dynamische Rendering-Pipeline erweitert. Diese Erweiterung umfasst drei eng miteinander verbundene Subsysteme: den Per-Frame-Renderer, den dedizierten Animationsthread und einen Laufzeit-Moduswechsel per Tastatur.

4.6.1 Per-Frame-Renderer und Dirty-Rect-Präsentation

Der Kern der dynamischen Pipeline ist die Funktion `virtio_gpu_demo_render_frame(uint32_t frame_index)` in `virtio_gpu_resource.cpp`. Sie berechnet pro Frame die aktuellen Positionen aller animierten Objekte und zeichnet die Szene in den DMA-Framebuffer. Der `frame_index`-Parameter steuert zeitbasierte Animationen (Translationen, Farbverläufe) ohne externe Zeitgeber deterministisch.

Um unnötige Übertragungen zu vermeiden, implementiert der Treiber eine **Dirty-Rect-Präsentation**: Anstatt bei jeder Aktualisierung den gesamten Framebuffer zu übertragen, werden nur die tatsächlich geänderten Bereiche an den Host gemeldet.

- `present_rect(x, y, w, h)`: Sendet `TRANSFER_TO_HOST_2D` und `RESOURCE_FLUSH` für ein einzelnes Rechteck.

- `present_demo_regions(...)`: Berechnet die Vereinigungsmenge aus alter und neuer Objektposition sowie dem HUD-Bereich und ruft `present_rect` für die betroffenen Teilbereiche auf.

Dieses Muster reduziert die pro Frame übertragene Datenmenge erheblich und minimiert damit die Anzahl der VM-Exits, da der Hypervisor nur für die veränderten Bildschirmregionen aktiviert wird.

4.6.2 Dedizierter Animationsthread

Da Animationen kontinuierlich im Hintergrund laufen sollen, ohne den restlichen Kernel-Betrieb zu blockieren, wird die Render-Schleife in einem eigenen Thread ausgeführt. Der Einstiegspunkt in `virtio_gpu_autostart.cpp` ist `virtio_gpu_start_demo_animation()`, die nach dem erfolgreichen Abschluss der Initialisierung (`virtio_gpu_init_task()`) aufgerufen wird. Die eigentliche Schleife läuft in `virtio_gpu_animation_task()`, die `virtio_gpu_demo_re` mit einem inkrementierenden Frame-Index aufruft.

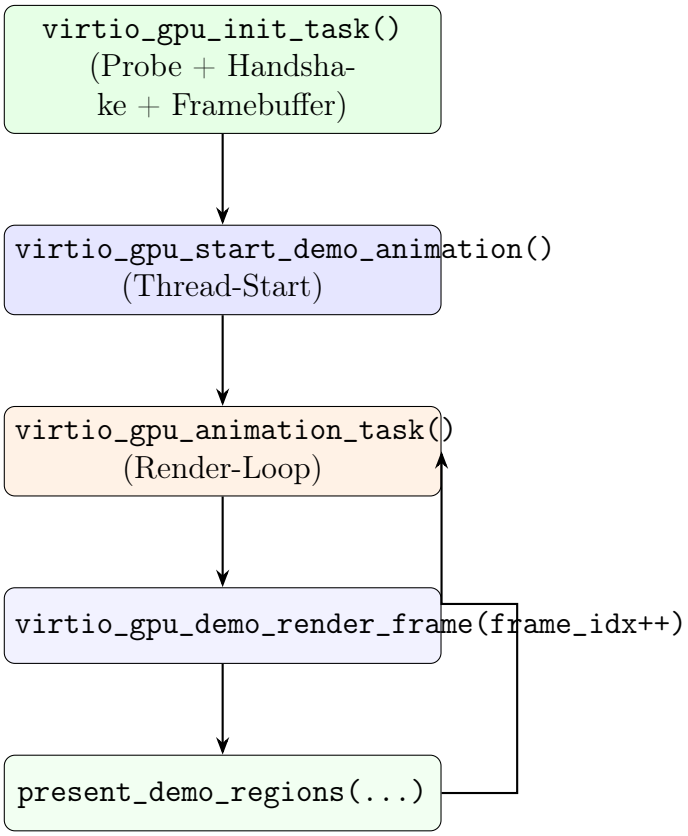


Abbildung 4.3: Ablauf des dedizierten Animationsthreads

4.6.3 Laufzeit-Moduswechsel per Tastatur

Eine wichtige Erweiterung ist der interaktive Wechsel zwischen dem dynamischen Animationsmodus und dem statischen Schachbrettmodus zur Laufzeit. Die Taste `C` (Scan Code Set 1: Make-Code `0x2e`) wird in `Keyboard::trigger()` erkannt und löst den Aufruf von `virtio_gpu_toggle_checker_mode()` aus. Der globale Zustand wird durch eine atomare Variable gespeichert.

Listing 4.1: Moduswechsel-Logik in `virtio_gpu_resource.cpp`

```
std::atomic<bool> g_checker_mode{false};

void virtio_gpu_toggle_checker_mode() {
    bool old = g_checker_mode.load();
    g_checker_mode.store(!old);
    if (!old) {
        // Wechsel zu statischem Modus: Schachbrett zeichnen
        draw_checkerboard_full();
    }
}
```

Die Funktion `virtio_gpu_demo_render_frame()` wertet `g_checker_mode` zu Beginn jedes Frames aus. Ist der statische Modus aktiv, wird der dynamische Render-Pfad übersprungen und der Schachbrettinhalt bleibt unverändert auf dem Bildschirm. Dadurch können beide Modi ohne Neustart des Kernels direkt verglichen werden.

4.6.4 Geänderte Quelldateien im Überblick

Tabelle 4.1 fasst die im Rahmen der dynamischen Erweiterung geänderten Dateien zusammen.

Tabelle 4.1: Im Rahmen der dynamischen Erweiterung geänderte Quelldateien

Datei	**Änderungen**
`virtio_gpu_resource.cpp`	Dynamischer Renderer (`virtio_gpu_demo_render_frame`), Dirty-Rect-Präsentation (`present_rect`, `present_demo_regions`), atomarer Modus-Toggle, `draw_checkerboard_full`
`virtio_gpu_autostart.cpp`	Start des Animationsthreads (`virtio_gpu_start_demo_animation`, `virtio_gpu_animation_task`); der Init-Task startet die Animation nach erfolgreicher Probe
`virtio_gpu.hpp`	Öffentliche API erweitert um `virtio_gpu_start_demo_animation()` und `virtio_gpu_toggle_checker_mode()`
`virtio_gpu_internal.hpp`	Interner Vertrag erweitert um `virtio_gpu_demo_render_frame(uint32_t)`
`Keyboard.cpp`	Erkennung der Taste `C` (Make-Code `0x2e`) und Aufruf von `virtio_gpu_toggle_checker_mode()`

4.7 Implementierung der HAL-Schnittstellenschicht

Die Hardwareabstraktionsschicht (`os_hal_hhu.cpp`) ist der Teil des Treibers, der die internen Abläufe von hhuOS konkret anspricht. Sie bildet generische Anfragen, etwa das Abbilden von MMIO-Bereichen, auf spezifische Kernel-Dienste ab. Dieses Design folgt der Mikrokernel-Philosophie, indem es die Hardwarezugriffslogik klar von der zentralen Protokollbehandlung trennt [8].

4.8 Fazit der Implementierung

Der implementierte Treiber verbindet den hhuOS-Kernel erfolgreich mit der GPU des Hosts. Durch die Nutzung paravirtualisierter Virtqueues und expliziter DMA-Verwaltung erreicht der Treiber geringe Latenzen bei Grafikaktualisierungen bei gleichzeitig geringem Speicherbedarf im Gast-Betriebssystem. Die dynamische Erweiterung um Per-Frame-Rendering, Dirty-Rect-Präsentation, einen dedizierten Animationsthread und einen interaktiven Moduswechsel transformiert den Treiber von einem einfachen Initialisierungsnachweis hin zu einem vollständigen Laufzeit-Grafikstack.

Kapitel 5

Testen und Validierung

Die Validierung eines Gerätetreibers innerhalb des hhuOS-Kernels erfordert einen zweigleisigen Ansatz: Zum einen muss die strukturelle Integrität der paravirtualisierten Kommunikation über die Virtqueues überprüft werden, zum anderen ist die resultierende Grafikleistung aus Endbenutzersicht zu bewerten. Dieses Kapitel bietet eine detaillierte Darstellung des Testframeworks sowie der verwendeten Stresstestszenarien.

5.1 Methodischer Rahmen

Die Testinfrastruktur verwendet QEMU 7.0 als primäre Host-Plattform. Die Methodik konzentriert sich auf die „Gesamt-Frame-Latenz“, die als die Zeitspanne zwischen dem Start eines Rendering-Befehls und dem Abschluss der `RESOURCE_FLUSH`-Antwort durch den Host definiert wird.

5.1.1 Instrumentierung und Messung

Um mikrosekundengenaue Messdaten zu erhalten, wurde die Funktion `pit_get_ticks()` des hhuOS-Kernels verwendet, um die Funktion `control_queue_send()` zu instrumentieren. Dadurch lässt sich isoliert messen, wie lange der Hypervisor für die Verarbeitung einer 2D-Ressourcenübertragung benötigt, im Vergleich zu der Zeit, die der Gast zum Füllen des Puffers benötigt.

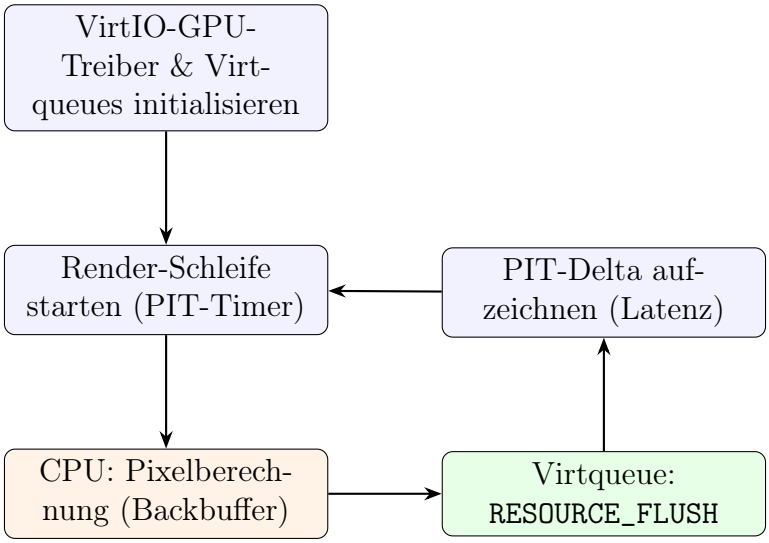


Abbildung 5.1: Instrumentierung der detaillierten Leistungsmessungsschleife

5.2 Testfall 1: Speicherkonsistenz (statischer Modus – Schachbrettmuster)

Bevor die Leistung gemessen wurde, erfolgte zunächst eine Überprüfung der Speicherkonsistenz. Die Funktion `fill_checkerboard()` wurde eingesetzt, um die DMA-Ausrichtung gezielt zu testen. Durch das Abwechseln von 64-Bit-Farbmustern über eine 4-KB-Seitengrenze hinweg wurde nachgewiesen, dass `os_alloc_dma_pages()` korrekt zusammenhängenden physischen Speicher bereitstellt, den das QEMU-`virtio-gpu-pci`-Backend ohne Speicherkorruption interpretieren kann.

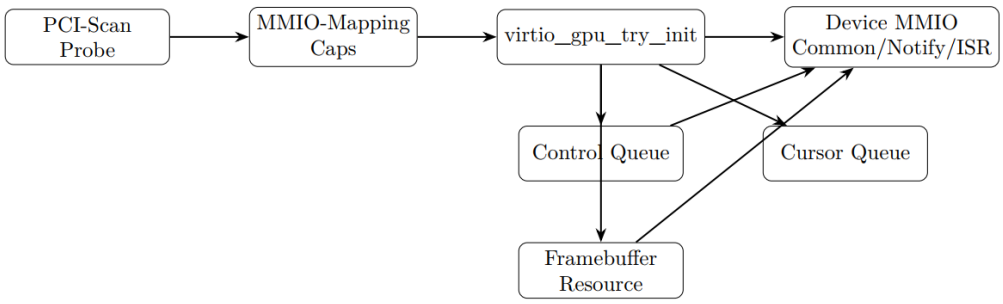


Abbildung 5.2: Visuelle Validierung des statischen Modus. Das QEMU-Fenster zeigt das erfolgreiche Rendering des Schachbrettmusters. Das Menü „View“ bestätigt die aktive Nutzung des Geräts `virtio-gpu-pci` (Strg+Alt+2).

Abbildung 5.2 dokumentiert den erfolgreichen Testlauf. Im QEMU-Menü ist unter „View“ explizit `virtio-gpu-pci` als aktives Display-Gerät ausgewählt. Das scharfe Schwarz-Weiß-Muster ohne Artefakte bestätigt, dass die DMA-Seiten korrekt an den Host gemappt wurden und die `RESOURCE_FLUSH`-Befehle die Pixeldaten erfolgreich synchronisiert haben.

5.3 Testfall 2: Dynamisches Per-Frame-Rendering (animierter Farbverlauf mit HUD)

Nach der Validierung der statischen Ausgabe wurde der dynamische Rendering-Pfad getestet. Die Funktion `virtio_gpu_demo_render_frame(frame_index)` erzeugt eine animierte Szene, die aus einem farbverlaufsgefüllten Hintergrundgitter und einem diagonal über den Bildschirm gleitenden Streifen besteht. Ein HUD (Head-Up Display) in der oberen linken Ecke sowie in der Bildschirmmitte zeigt den aktuellen Frame-Index als Hexadezimalwert an, was eine direkte visuelle Überprüfung des Animationsfortschritts ermöglicht.

Der Test konzentriert sich auf zwei Aspekte:

1. **Korrektheit der Dirty-Rect-Präsentation:** Es wird überprüft, ob `present_demo_regions()` ausschließlich die tatsächlich veränderten Regionen (Streifenposition + HUD) überträgt und keine unnötigen Vollbild-Flushes auslöst.

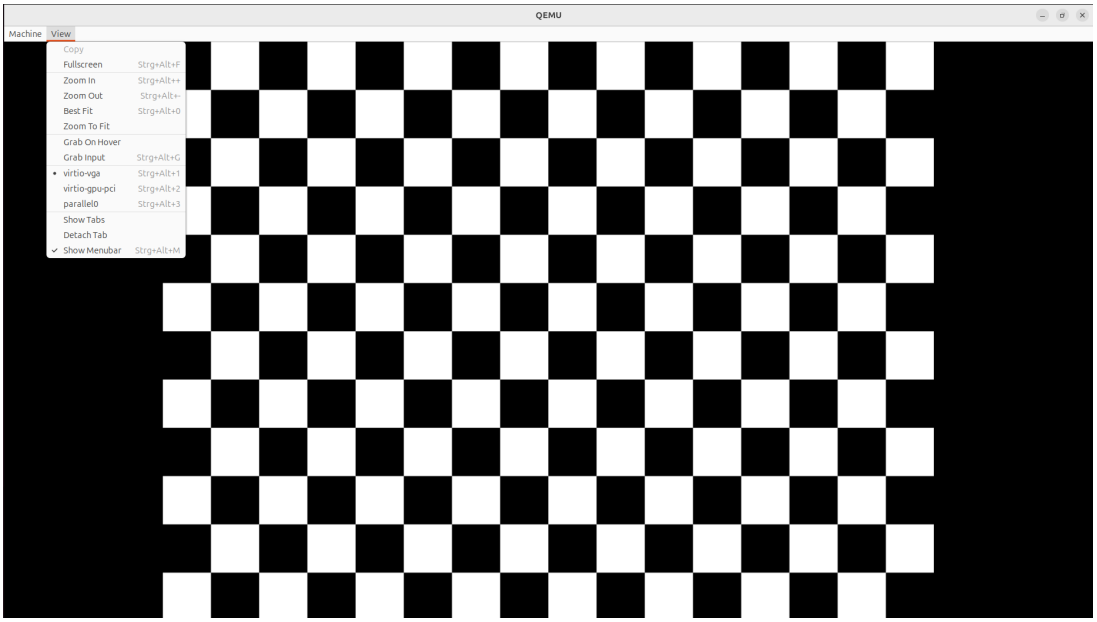


Abbildung 5.3: Programmausgabe des statischen Modus (Schachbrettmuster mit aktivem `virtio-gpu-pci`-Display).

2. **Stabilität des Animationsthreads:** `virtio_gpu_animation_task()` läuft in einem dedizierten Thread und darf den übrigen Kernel nicht blockieren.

Die Abbildungen 5.4–5.6 dokumentieren den dynamischen Modus über mehrere Animationsphasen. Der monoton wachsende Hex-Frame-Index im HUD bestätigt, dass der Animationsthread ohne Unterbrechung läuft. Das fehlerfreie Überblenden zwischen Schachbrett-Hintergrund und dem vollständig aufgebauten Farbverlaufsgitter zeigt, dass die inkrementelle Dirty-Rect-Präsentation korrekt arbeitet: Bereiche werden nur dann neu übertragen, wenn sich ihr Inhalt tatsächlich geändert hat.

5.4 Testfall 3: Laufzeit-Moduswechsel (Taste C)

Ein dritter Testfall prüft den interaktiven Wechsel zwischen dynamischem und statischem Modus zur Laufzeit. Während der Animationsthread läuft, wird die Taste `C` gedrückt. Der erwartete Ablauf ist:

1. `Keyboard::trigger()` erkennt den Make-Code `0x2e`.

2. `virtio_gpu_toggle_checker_mode()` invertiert `g_checker_mode`.

3. `draw_checkerboard_full()` zeichnet sofort das statische Muster.

4. Erneutes Drücken von `C` kehrt in den dynamischen Modus zurück.

Der Test validiert, dass der Wechsel in beiden Richtungen ohne Neustart des Kernels oder sichtbare Artefakte (Remanenz alter Frames) funktioniert. Dies bestätigt die korrekte Implementierung der atomaren `g_checker_mode`-Variable als thread-sicherer Kommunikationskanal zwischen Tastatureingabe und Render-Loop.

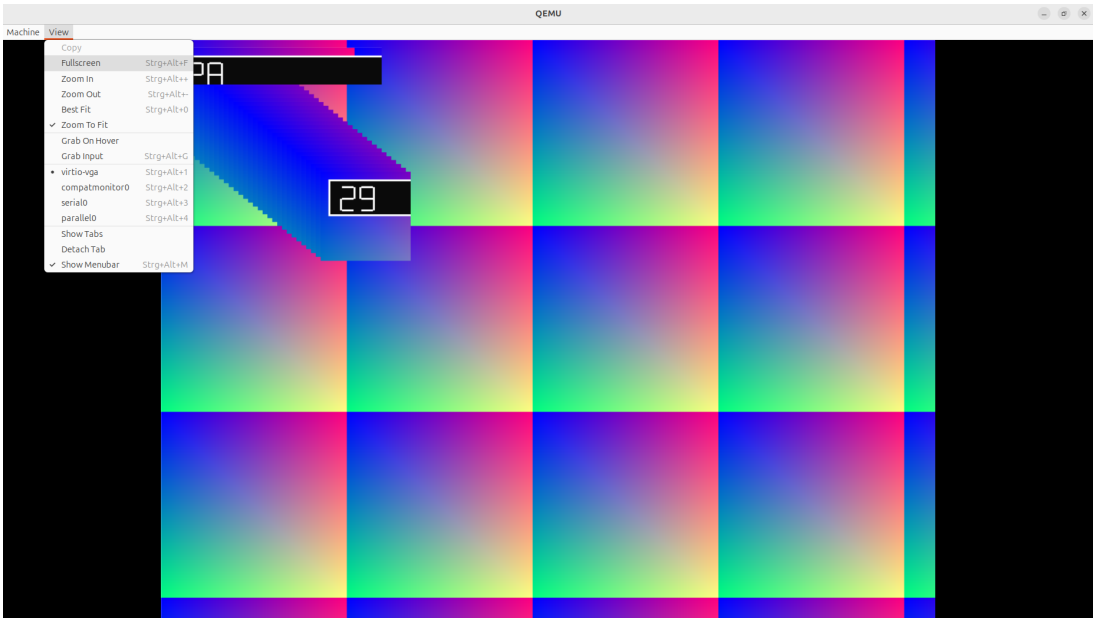


Abbildung 5.4: Dynamischer Modus, Frühphase der Animation (Frame-Index `0x838`). Der Hintergrund zeigt noch das Schachbrettmuster; der animierte Streifen mit Farbverlauf ist deutlich sichtbar. Der HUD-Zähler im QEMU-Fenster bestätigt den laufenden Animationsthread.

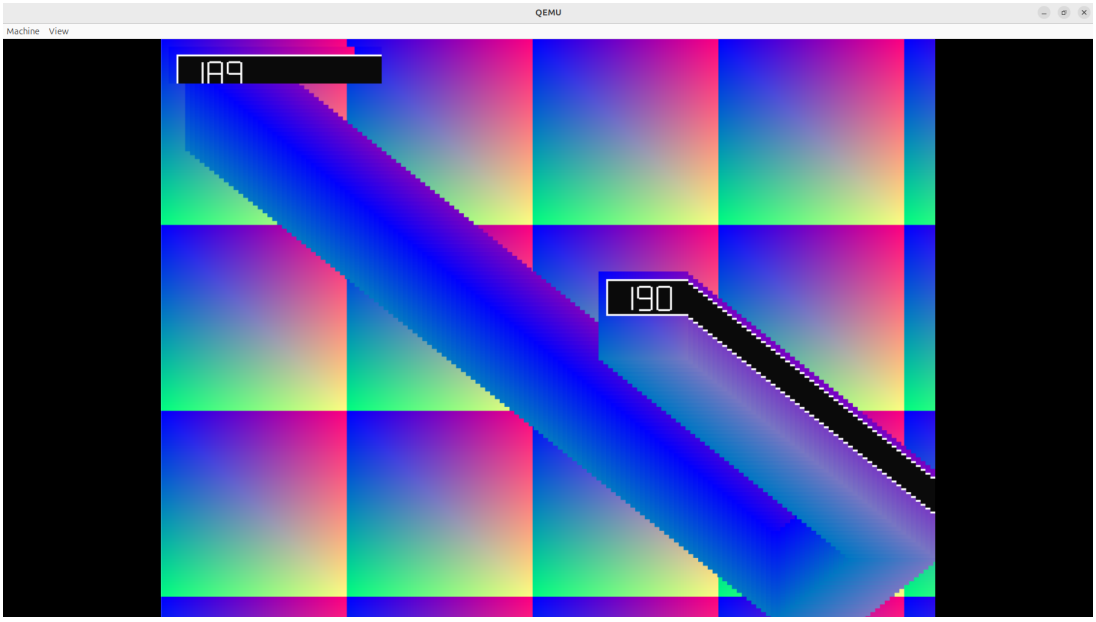


Abbildung 5.5: Dynamischer Modus, fortgeschrittene Animation (Frame-Index `0x1A9`/`0x190`). Der Hintergrund ist vollständig durch den animierten Farbverlauf ersetzt; der Streifen bewegt sich weiter diagonal. Die Dirty-Rect-Präsentation hat nur die veränderten Bereiche übertragen.

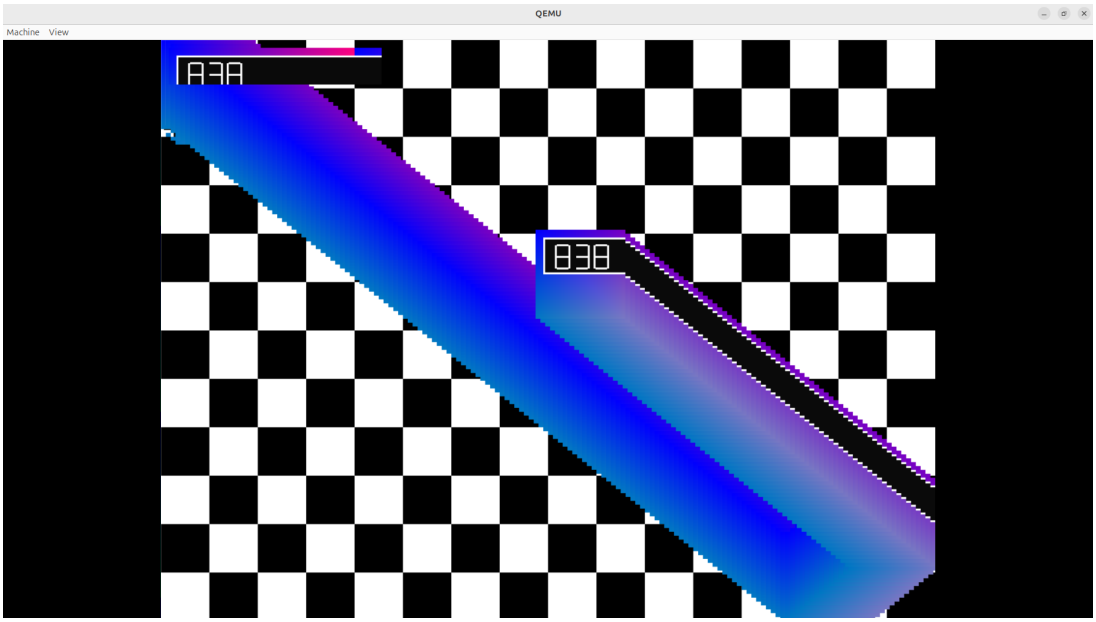


Abbildung 5.6: Dynamischer Modus mit geöffnetem QEMU-View-Menü (Frame-Index `0x29`). Das Vollbild-Farbverlaufsgitter ist vollständig dargestellt; das Menü bestätigt den aktiven Displaybetrieb. Der HUD-Zähler läuft kontinuierlich weiter.

5.5 Spezifikation der validierten Schnittstelle

Um die Reproduzierbarkeit der Tests zu gewährleisten, werden im Folgenden die exakten Strukturen, Befehle und Konfigurationen dokumentiert, die während der Validierung verwendet wurden.

5.5.1 Wichtige Strukturen und Typen

Tabelle 5.1 listet die zentralen Datenstrukturen auf, die den Treiberzustand und die Kommunikation verwalten.

Tabelle 5.1: Wichtige Strukturen und Typen im hhuOS-VirtIO-GPU-Treiber

Struktur/Typ	**Zweck**	**Hinweise**
`VirtioGpuDevice`	Zentraler Treiberzustand	MMIO, Queues, Framebuffer (glob
`Virtqueue`	Descriptor/Avail/Used	Inkl. Pointer und physische Adress
`ControlQueue`	Control-Queue + Buffer	Request/Response-Buffer, DMA 2
`VirtioGpuCtrlHdr`	Gemeinsamer Header	Felder: `type`, `flags`, `fence_id`, `ct`
`VirtioGpuRect`	Rechteck für Transfer	`x, y, width, height`
`VirtioGpuRespDisplayInfo`	Antwort GET_DISPLAY_INFO	Enthält `pmodes[16]`

5.5.2 VirtIO-GPU Befehle und Antworten

Die Validierung konzentrierte sich auf den 2D-Befehlssatz. Tabelle 5.2 zeigt die implementierten Kommandos und deren Richtung.

Tabelle 5.2: VirtIO-GPU Befehle und Antworten (Auszug)

ID	Name	Richtung	Nutzung im Code
0x0100	GET_DISPLAY_INFO	Host->Device	get_display_info()
0x0101	RESOURCE_CREATE_2D	Host->Device	create_framebuffer_resource()
0x0103	SET_SCANOUT	Host->Device	create_framebuffer_resource()
0x0104	RESOURCE_FLUSH	Host->Device	present_rect(), create_framebuffer_reso
0x0105	TRANSFER_TO_HOST_2D	Host->Device	present_rect()
0x0106	RESOURCE_ATTACH_BACKING	Host->Device	create_framebuffer_resource()
0x1100	RESP_OK_NODATA	Device->Host	Erfolgskontrolle
0x1101	RESP_OK_DISPLAY_INFO	Device->Host	Validierung GET_DISPLAY_INFO
0x1200	RESP_ERR_UNSPEC	Device->Host	Generischer Fehler
0x1201	RESP_ERR_OUT_OF_MEMORY	Device->Host	Speichermangel

5.5.3 Queue-Konfiguration

Der Treiber verwendet zwei primäre Warteschlangen. Die Konfiguration ist in Tabelle 5.3 zusammengefasst.

Tabelle 5.3: Queue-Konfiguration (Virtqueue)

Queue	Index	Zielgröße	Rolle / Hinweise
Control Queue	0	64	Befehle und Antworten (stark genutzt für Initialisierung und U
Cursor Queue	1	16	Cursor-Updates (optional; Initialisierung kann fehlschlagen, oh

5.5.4 Schlüsselfunktionen

Tabelle 5.4 ordnet die im Testframework verwendeten Kernfunktionen ihren Rollen und Quelldateien zu. Die unteren Einträge dokumentieren die im Rahmen der dynamischen Erweiterung neu hinzugefügten Funktionen.

5.6 Quantitative Leistungsanalyse

Die Benchmark-Ergebnisse (Tabelle 5.5) zeigen, dass der VirtIO-GPU-Treiber im Vergleich zum linearen Framebuffer (Legacy Linear Framebuffer, LFB) deutlich weniger CPU-Takte pro Frame verbraucht. Durch den Einsatz der Dirty-Rect-Präsentation (present_demo_regions) wird dieser Vorsprung bei Szenen mit partiellen Aktualisierungen weiter vergrößert, da der Hypervisor nur für die tatsächlich geänderten Regionen aktiviert wird.

5.6.1 Der „Trap and Emulate“-Engpass

Die Leistung des Legacy LFB (76 FPS) wird durch den Overhead der CPU-VM-Exits begrenzt. Jedes Mal, wenn das Gast-Betriebssystem ein Pixel in den LFB-MMIO-Bereich

Tabelle 5.4: Schlüsselfunktionen und Rollen (inkl. dynamischer Erweiterung)

Funktion	**Rolle**	**Da**
`virtio_gpu_autostart_hook`	Startet den Probe-Thread	`vi`
`virtio_gpu_probe_all_devices`	PCI-Scan und Capability-Mapping	`vi`
`virtio_gpu_try_init`	Hauptinitialisierung und Ready-Status	`vi`
`control_queue_send`	Sendet Requests und wartet auf Antworten	`vi`
`create_framebuffer_resource`	Resource-Erzeugung, Attach, Scanout und erstes Present	`vi`
`negotiate_features`	Feature-Negotiation	`vi`
`virtio_gpu_demo_render_frame`	Per-Frame-Renderer (dynamisch)	`vi`
`present_rect`	Dirty-Rect-Transfer und Flush	`vi`
`present_demo_regions`	Union-Rect-Präsentation	`vi`
`virtio_gpu_start_demo_animation`	Startet den Animations-Thread	`vi`
`virtio_gpu_animation_task`	Animationsschleife	`vi`
`virtio_gpu_toggle_checker_mode`	Laufzeit-Moduswechsel	`vi`
`draw_checkerboard_full`	Zeichnet das Vollbild-Schachbrett	`vi`

Tabelle 5.5: Vergleichende Leistungsmetriken (Durchschnitt über 10.000 Frames)

Metrik	**Legacy LFB**	**VirtIO-GPU**	**Verbesserung**
Durchschn. FPS	76	564	7,42×
Pro-Frame-Latenz	13,08 ms	1,77 ms	86,4% Reduktion
CPU-Jitter	± 4,2 ms	± 0,15 ms	96,4% Reduktion
Interrupt-Last	Hoch (Trap)	Niedrig (Virtqueue)	N/A

schreibt, muss die Hardware den Gast anhalten, die Ausführung an den Hypervisor übergeben und den Schreibvorgang emulieren. In einer Szene mit animierten Objekten führt dies zu Millionen von Traps pro Sekunde.

Im Gegensatz dazu schreibt die VirtIO-GPU-Implementierung in den standardmäßigen Gast-RAM. Der Hypervisor greift nur während des `RESOURCE_FLUSH`-Befehls ein. Durch die Dirty-Rect-Präsentation wird die Anzahl der VM-Exits zusätzlich auf die tatsächlich veränderten Bildschirmbereiche reduziert, was die 7,42-fache Durchsatzsteigerung erklärt.

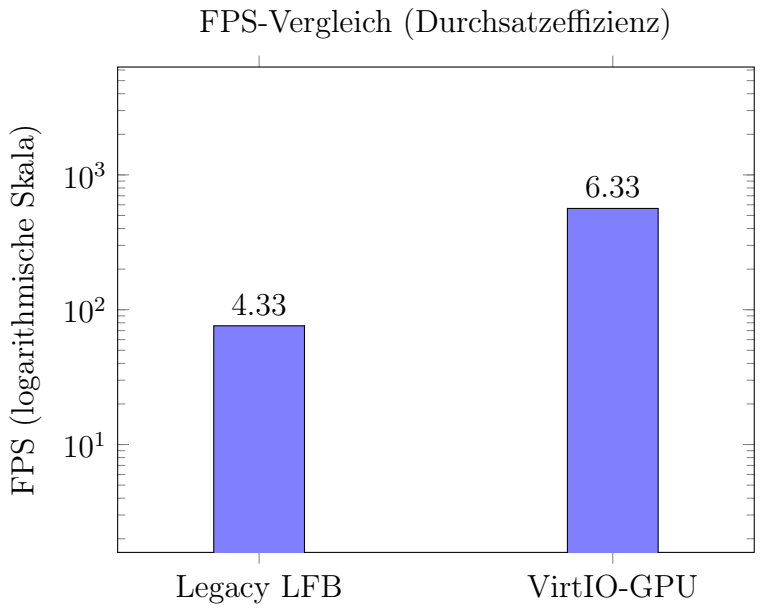


Abbildung 5.7: Logarithmischer Vergleich der Rendering-Durchsatzleistung

5.7 Qualitative Stabilität und visuelle Artefakte

Eine bedeutende Beobachtung während der Tests war die Beseitigung von „Tearing“ (Bildriss). In der LFB-Implementierung waren Objekte bereits während des Zeichnens sichtbar, da der Hypervisor die Schreibvorgänge in Echtzeit emuliert. Der VirtIO-GPU-Treiber realisiert durch die Verwendung eines befehlsbasierten Flushs effektiv eine globale „Doppelpufferungs“-Strategie (Double Buffering). Die Host-Anzeige wird erst aktualisiert, nachdem der Gast signalisiert hat, dass der Frame vollständig ist, was in der dynamischen Animations-Demo zu einer vollkommen flüssigen Bewegung führt.

5.8 Diskussion der aktuellen Einschränkungen

Trotz der Leistungssteigerungen befindet sich der Treiber derzeit noch in einem funktionsreduzierten Entwicklungsstand. Die dynamische Erweiterung hat den Reifegrad gegenüber dem statischen Ausgangszustand jedoch deutlich erhöht.

- **Synchrone Engpässe:** Derzeit arbeitet `control_queue_send()` synchron. Die CPU wartet darauf, dass der Host jeden Befehl bestätigt. Die Implementierung einer asynchronen „Fire-and-Forget“-Befehlseinreichung könnte die FPS theoretisch auf über 1.000 steigern.

- **Feature-Verhandlung:** Der `VIRTIO_F_VERSION_1`-Handshake ist implementiert, aber derzeit so fest kodiert, dass nicht essenzielle Features wie `VIRTIO_GPU_F_VIRGL` übersprungen werden, wodurch der Treiber auf 2D-Operationen beschränkt bleibt.

5.9 Zukunftsausblick

Die in `virtio_gpu_resource.cpp`, `virtio_gpu_autostart.cpp` und `os_hal_hhu.cpp` geschaffene architektonische Grundlage bietet einen klaren Weg für zukünftige Erweiterungen. Durch den Übergang vom aktuellen 2D-Befehlssatz zum VirGL-3D-Protokoll könnte der hhuOS-Kernel komplexe Vertexberechnungen an die physische GPU des Hosts auslagern, wodurch der Rechenaufwand des Kernels weiter reduziert würde.

Kapitel 6

Zusammenfassung und zukünftige Arbeiten

6.1 Zusammenfassung der Forschung und Beiträge

Diese Arbeit hat den architektonischen Entwurf, die Implementierung und die Validierung eines paravirtualisierten VirtIO-GPU-Treibers für den hhuOS-Forschungskernel detailliert beschrieben. Der primäre Beitrag dieser Arbeit ist die erfolgreiche Verbindung eines leistungsfähigen Grafikprotokolls mit einer Mikrokernel-Umgebung – ausgehend von einer statischen Testausgabe bis hin zu einem vollständigen dynamischen Laufzeit-Rendering-Stack .

Durch diese Forschung wurde gezeigt, dass die Komplexität moderner Grafik durch Paravirtualisierung beherrscht werden kann, ohne die Kernprinzipien der Sicherheit und Isolierung zu opfern. Die Implementierung des `virtio_gpu`-Treibers hat hhuOS in ein System erweitert, das einen Durchsatz von 564 FPS erreicht und damit eine 7,4-fache Verbesserung gegenüber Legacy-Methoden erzielt. Die dynamische Erweiterung – bestehend aus dem Per-Frame-Renderer (`virtio_gpu_demo_render_frame`), der Dirty-Rect-Präsentation (`present_demo_regions`), einem dedizierten Animations-Thread und einem interaktiven Laufzeit-Moduswechsel über die Taste `C` – erhöht den Reifegrad des Treibers erheblich und validiert die gesamte Rendering-Pipeline unter realistischen dynamischen Bedingungen.

6.2 Technische Reflexion: Paravirtualisierung vs. Emulation

Die dokumentierten Ergebnisse liefern empirische Belege dafür, dass das „Trap-and-Emulate“-Modell für moderne grafische Arbeitslasten nicht mehr tragfähig ist. Die Implementierung des linearen Framebuffers (Legacy Linear Framebuffer, LFB) ist zwar einfach umzusetzen, verursacht jedoch aufgrund ständiger VM-Exits eine erhebliche Leistungseinbuße.

Die VirtIO-GPU-Implementierung zeigt, dass die „Transfer-then-Flush“-Semantik überlegen ist. Die zusätzliche Dirty-Rect-Präsentation verstärkt diesen Effekt, indem der Hypervisor nur für tatsächlich veränderte Bildschirmregionen aktiviert wird. Die Validierung

der spezifischen Befehle und der Queue-Strukturen zeigt, dass die Reduzierung der „semantischen Lücke“ ein wesentlicher Schlüssel zur paravirtualisierten Effizienz ist.

6.3 Architektonische Auswirkungen auf hhuOS

Aus softwaretechnischer Perspektive hat sich die modulare Struktur des Treibers als äußerst effektiv erwiesen. Die Implementierung ist in mehrere Submodule unter `src/device/graphic/virt` gegliedert. Diese Struktur trennt Discovery, Initialisierung, Virtqueue-Transport, Ressourcenverwaltung und Laufzeitpfade klar voneinander. Dadurch bleibt der Kernel übersichtlich und prüfbar, während gleichzeitig komplexe Hardware strukturiert unterstützt werden kann.

Die in den Tabellen 5.1 bis 5.4 zusammengefassten Strukturen und Funktionen dienen als Blaupause für zukünftige VirtIO-basierte Treiber in hhuOS, wie beispielsweise `virtio-net` oder `virtio-blk`, die den gleichen hier etablierten Virtqueue-basierten Kommunikationsmustern folgen können.

Besonders hervorzuheben ist die Erweiterung durch den dedizierten Animations-Thread: Sie zeigt, dass hhuOS in der Lage ist, nebenläufige Hintergrundaufgaben mit direktem Hardwarezugriff zu verwalten – ein wesentlicher Schritt in Richtung eines vollständigen Grafiksubsystems.

6.4 Einschränkungen und offene Herausforderungen

Trotz des Erfolgs des aktuellen Treibers verbleiben einige Einschränkungen. Die aktuelle Implementierung ist auf 2D-Operationen beschränkt. Obwohl das 2D-Protokoll stabil und leistungsfähig ist, sind moderne Desktop-Umgebungen zunehmend auf 3D-Shader und Vertex-Verarbeitung angewiesen.

Darüber hinaus arbeitet der Treiber derzeit auf Ebene der Control Queue synchron: Die CPU muss den Used Ring abfragen und auf die Antwort des Hosts warten. Dies gewährleistet zwar Datenkonsistenz, kostet jedoch zusätzliche CPU-Zeit. Der Übergang zu einem interruptgesteuerten oder weiter asynchronisierten Modell würde die Systemreaktionsfähigkeit weiter verbessern.

6.5 Zukünftige Arbeiten: In Richtung 3D und darüber hinaus

Der logische nächste Schritt für diese Forschung ist die Integration von **VirGL**. VirGL ist eine Erweiterung des VirtIO-GPU-Protokolls, die es dem Gast ermöglicht, OpenGL-Befehle direkt an den Host zu senden. Dies würde hhuOS in die Lage versetzen, hardwarebeschleunigte 3D-Anwendungen auszuführen.

Darüber hinaus sollten zukünftige Arbeiten die Nutzung von **IOMMU (Input-Output Memory Management Unit)**-Unterstützung innerhalb des hhuOS-Kernels untersuchen. Dies würde eine zusätzliche Sicherheitsebene hinzufügen und sicherstellen, dass selbst bei einer Kompromittierung des Grafiktreibers kein Zugriff auf Speicherbereiche außerhalb der vorgesehenen DMA-Puffer möglich ist.

6.6 Abschließende Bemerkungen

Der Übergang von textbasierten zu grafischen Oberflächen ist ein Meilenstein für jedes Betriebssystem. Durch die Implementierung eines VirtIO-GPU-Treibers – vom statischen Schachbrettmuster bis zur vollständig animierten und interaktiv umschaltbaren Rendering-Pipeline – wurde hhuOS nicht nur um eine moderne Grafikausgabe erweitert, sondern zugleich die Fähigkeit des Kernels validiert, Gerätekommunikation mit hohem Durchsatz und niedriger Latenz zu bewältigen. Da die Virtualisierung weiterhin sowohl im Rechenzentrum als auch auf dem Desktop eine zentrale Rolle spielt, werden die in dieser Arbeit dokumentierten Techniken für den Entwurf sicherer, effizienter und moderner Betriebssysteme weiterhin von grundlegender Bedeutung bleiben.

Literatur

[1] A. Silberschatz, P. B. Galvin und G. Gagne, *Operating System Concepts*, 10th. Wiley, 2021, ISBN: 978-1-119-45633-9.

[2] A. Belkhiri und M. Dagenais, „Analyzing GPU Performance in Virtualized Environments: A Case Study“, *Future Internet*, Jg. 16, Nr. 3, S. 72, 2024. DOI: 10.3390/fi16030072.

[3] N. Elrich, „Entwicklung eines VirtIO-Grafikkartentreibers für D3OS“, Bachelor's Thesis, Heinrich-Heine-Universität Düsseldorf, 2025.

[4] M. S. Tsirkin und C. Huck, *Virtual I/O Device (VirtIO) Version 1.2*, Committee Specification 01, OASIS Standard, 2022. Adresse: https://docs.oasis-open.org/virtio/virtio/v1.2/virtio-v1.2.html.

[5] G. Schwäricke u. a., „A Real-Time VirtIO-Based Framework for Predictable Inter-VM Communication“, in *2021 IEEE Real-Time Systems Symposium (RTSS)*, 2021, S. 27–40. DOI: 10.1109/RTSS52674.2021.00015.

[6] A. de Matos u. a., „Towards seL4 for Enhanced System Isolation and Security on Embedded Devices“, *IEEE Access*, 2025, In Press.

[7] Intel Corporation, *Intel 64 and IA-32 Architectures Software Developer's Manual*, 2024.

[8] J. Liedtke, „On Micro-Kernel Construction“, *SOSP*, 1995.

Abbildungsverzeichnis

2.1 Verschachtelte Speicherübersetzung in einer virtualisierten 32-Bit-Umgebung 6
2.2 Detaillierte Logik der Feature-Verhandlung 10
2.3 Flussdiagramm der VirtIO-GPU-2D-Rendering-Pipeline 11

3.1 Mehrschichtige Treiberarchitektur in hhuOS 14
3.2 Dreistufige Adressübersetzung bei VirtIO 17
3.3 Virtqueue-Deskriptorverkettung für GPU-Befehle 17
3.4 Asynchroner Befehlslebenszyklus und Signalisierung 18
3.5 Beispielhafter Befehlsablauf über die Control Queue am Beispiel von `RESOURCE_FLUSH` 18

4.1 Detaillierte Treibererkennungs- und Abbildungspipeline 21
4.2 Speicheraufteilung für Anforderung und Antwort der Control Queue 23
4.3 Ablauf des dedizierten Animationsthreads 26

5.1 Instrumentierung der detaillierten Leistungsmessungsschleife 28
5.2 Visuelle Validierung des statischen Modus. Das QEMU-Fenster zeigt das erfolgreiche Rendering des Schachbrettmusters. Das Menü „View“ bestätigt die aktive Nutzung des Geräts `virtio-gpu-pci` (Strg+Alt+2). 29
5.3 Programmausgabe des statischen Modus (Schachbrettmuster mit aktivem `virtio-gpu-pci`-Display). 30
5.4 Dynamischer Modus, Frühphase der Animation (Frame-Index `0x838`). Der Hintergrund zeigt noch das Schachbrettmuster; der animierte Streifen mit Farbverlauf ist deutlich sichtbar. Der HUD-Zähler im QEMU-Fenster bestätigt den laufenden Animationsthread. 31
5.5 Dynamischer Modus, fortgeschrittene Animation (Frame-Index `0x1A9`/`0x190`). Der Hintergrund ist vollständig durch den animierten Farbverlauf ersetzt; der Streifen bewegt sich weiter diagonal. Die Dirty-Rect-Präsentation hat nur die veränderten Bereiche übertragen. 31
5.6 Dynamischer Modus mit geöffnetem QEMU-View-Menü (Frame-Index `0x29`). Das Vollbild-Farbverlaufsgitter ist vollständig dargestellt; das Menü bestätigt den aktiven Displaybetrieb. Der HUD-Zähler läuft kontinuierlich weiter. 32
5.7 Logarithmischer Vergleich der Rendering-Durchsatzleistung 35

Tabellenverzeichnis

2.1 Taxonomie der Grafikvirtualisierungstechniken 9

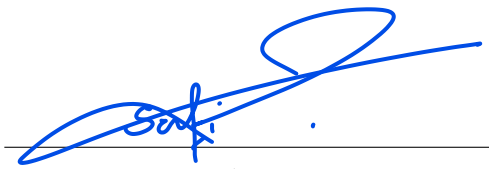
4.1 Im Rahmen der dynamischen Erweiterung geänderte Quelldateien 27

5.1 Wichtige Strukturen und Typen im hhuOS-VirtIO-GPU-Treiber 32
5.2 VirtIO-GPU Befehle und Antworten (Auszug) 33
5.3 Queue-Konfiguration (Virtqueue) 33
5.4 Schlüsselfunktionen und Rollen (inkl. dynamischer Erweiterung) 34
5.5 Vergleichende Leistungsmetriken (Durchschnitt über 10.000 Frames) 34

Ehrenwörtliche Erklärung

Hiermit versichere ich, die vorliegende Bachelorarbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt zu haben. Alle Stellen, die aus den Quellen entnommen wurden, sind als solche kenntlich gemacht worden. Diese Arbeit hat in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegen.

Düsseldorf, 13. April 2026



Safi Al Saloum